Table of Contents

- Table of Contents
- Database Topics
  - DBMS
  - Data Integrity Constraints
    - Entity Integrity
    - Referential Integrity
      - Referential Integrity (CASCADE, RESTRICT)
    - Domain Integrity
    - User-Defined Integrity
      - Attribute-Based Constraints vs Tuple-Based Constraints
  - Aggregation Functions
    - GROUP BY
    - GROUP BY Multiple Parameters
  - Joins
    - Natural Join
    - Join Using Specific Columns
- Normalization and Entity Relations
  - 1st Normal Form (1NF):
    - Non-normalized table:
  - 2nd Normal Form (2NF):
    - Non-normalized table:
  - 3rd Normal Form (3NF):
  - Entity Relationships:
  - Entity Relation Diagram
  - Normalizing with Entity Relationship Diagramming
  - Entity Diagrams Vs Relationship Diagrams
- T-SQL (Selects, Joins, Conditions)
  - T-SQL vs SQL
    - Variables:
    - Flow control
    - Changes to DELETE and UPDATE statements
- Indexes
  - Types of Indexes
    - 1. Single-Column Indexes
    - 2. Composite (Concatenated) Indexes
    - 3. Unique Indexes
    - 4. B-Tree Indexes
    - 5. Hash Indexes
    - 6. Clustered indexes
    - 7. Non-Clustered Indexes
      - Heap
      - Table with a Clustered Index
    - 8. Filtered Indexes

- Stored Procedures (SPs)
- Views:
    - Modification to View Using Triggers
    - Materialized views
        - Drawbacks of Updating Materialized Views:
    - Views vs. Materialized Views
- Functions
    - Table-Valued Functions
    - Key Differences Between Views and Functions
- Triggers
    - Different Trigger Types in SQL Server
        - DDL Triggers
        - DML Triggers
        - Logon Triggers
- Transactions
    - Properties of Transaction - ACID
    - Isolation Levels Topics
        - Dirty Read
        - Non-Repeatable Read
        - Phantom Read
            - Phantom reads Vs non-repeatable reads
    - Isolation Levels
- Comparison of Database Objects
- Temp Tables
    - Local Temporary Table:
    - Global Temporary Table
- CTEs (Common Table Expressions)
    - How to create a CTE
    - Recursive CTEs and Hierarchical Data
        - anchor_clause
        - recursive_clause
- Cursors
- Dynamic Queries
    - Dynamic SQL for Dynamic Filtering
- Database Tunning
    - Techniques
        - How does Database Performance Tuning benefit businesses?
        - Simple Performance Tuning Techniques
            - Handling Indexes
            - Optimizing Aggregation
            - Using the Right Data Types:
            - Dealing with Complex Joins and Multiple Table Relationships:
            - Balancing Normalization and Denormalization:
    - Auto-growth
        - Why set up the auto-growth for each database?

        - What is the purpose of auto-growth and it's consequences of not setting up the auto-growth properly?
        - How can I avoid auto-growth events from occurring too many times?
    - What is an index?
    - What is clustered index and non-clustered index?
    - WHAT IS A NON CLUSTERED INDEX
    - B-tree
    - SQL Page
        - Types of SQL pages
        - Page Splitting
        - Fill Factor
            - What is the significance of understanding the fill factor?
            - If we know that a fill factor of 100 can cause page splits, then what is the optimum number we should set for fill factor?
    - Query Optimizer
        - How does the execution plan select the most efficient plan to execute?
        - Reading the Execution Plan
        - Tool tip and its meaning
        - Table scan (Heap) Vs Clustered Index Scan
        - A CLUSTERED INDEX
        - NON CLUSTERED INDEX GUIDE
        - COVERED INDEX
        - OLAP (Online Analytical Processing) and OLTP (Online Transaction Processing) in databases:
            - OLAP:
            - OLTP:
    - Filtered Indexes
    - The SORT operator
    - Joins in execution plan
    - Statistics
    - Locking
        - Types of Lock Modes
        - Blocking
        - Deadlock
    - Data defragmentation
        - Why Data Fragmentation Occurs:
        - Benefits of Data Defragmentation:
        - Defragmentation steps
    - Rebuilding and reorganizing indices
        - Shrink or Compact the Database:
        - Update Statistics:
        - Partitioning Tables:
        - Regular Maintenance Plans:
        - Tools for Defragmentation:
            - Automatic Workload Repository (AWR)
    - Wait Statistics

        - Wait types
    - Compilation and Recompilation
        - The purpose of the Query Optimizer
        - Example of a sproc compiling and execution plan
        - Recompilation
        - What is the plan cache?
    - Tempdb configuration
        - Setting up tempdb
        - File number rule: Tempdb files based on number of CPU cores (SQL 2016)
- Ranking and Partitioning
    - What Is RANK()?
    - PARTITION BY
    - Example
    - ROW_NUMBER vs RANK vs DENSE_RANK
        - ROW_NUMBER()
        - RANK()
        - DENSE_RANK()

The `SEQUENCE` strategy uses a database sequence object to generate primary key values. This strategy is most commonly used with databases that support sequences, like PostgreSQL and Oracle.

Database Topics

DBMS

A Database Management System (DBMS) is software that manages databases. It provides tools for data storage, querying, updating, and administration. Examples include MySQL, PostgreSQL, Oracle, etc.

Data Integrity Constraints

Data integrity in a database ensures the accuracy, consistency, and reliability of data over its lifecycle. As you mentioned, integrity constraints are rules that help enforce this data integrity, and they can be categorized into several types. Here's a breakdown of the four main types of integrity constraints:

Entity Integrity

- Entity integrity ensures that each table in a relational database has a primary key, and that the values in this key are unique and not null.
- Importance: This constraint is crucial for identifying individual records uniquely, ensuring that there are no duplicate rows in a table.

Referential Integrity

Referential integrity ensures that relationships between tables remain consistent. Specifically, it requires that foreign key values in one table must match primary key values in another table or be null.

- Importance: This constraint maintains the logical relationships between tables, preventing orphan records and ensuring that relationships are valid.

- Example: In an orders table, the customer_id may be a foreign key that refers to the customer_id in a customers table. If a foreign key value in orders does not match any primary key in customers, or if it refers to a non-existent customer, it violates referential integrity.

Referential Integrity (CASCADE, RESTRICT)

Referential integrity ensures that relationships between tables remain consistent. Actions like CASCADE and RESTRICT determine how related rows behave on DELETE or UPDATE.

- CASCADE: Deletes or updates related rows.
- RESTRICT: Prevents the action if related rows exist.

```
CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    customer_id INT,
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id) ON DELETE
CASCADE;
);
```

- In this case, if a customer is deleted, all their orders are also deleted.
- Each of these concepts plays a critical role in ensuring that databases remain efficient, consistent, and scalable.

Domain Integrity

- Domain integrity specifies that all columns in a table must adhere to defined data types and allowable values (domains). A domain represents a set of valid values for a given attribute.
- Importance: This constraint ensures that data entered into the database is of the correct type and within acceptable limits, enhancing data quality and consistency.
- Example: If a column age in a users table is defined as an integer with a domain of 0 to 120, any entry outside this range would violate domain integrity.

User-Defined Integrity

- User-defined integrity refers to specific business rules or constraints that are defined by the user and do not fit into the categories of entity, domain, or referential integrity. These rules can be unique to the application or organization. Importance: This allows for customized constraints that can enforce business logic, providing flexibility to cater to specific application needs.
- Example: A rule might state that a loan record cannot be created if the borrower's credit score is below a certain threshold. This rule does not relate directly to primary keys, foreign keys, or domains, but it is vital for maintaining business rules.

```
CREATE TABLE borrowers (
    borrower_id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    credit_score INT CHECK (credit_score >= 300 AND credit_score <= 850)
```

```
-- Valid credit score range
);
```

Attribute-Based Constraints vs Tuple-Based Constraints

- Attribute-Based Constraints: These are applied to individual columns, like CHECK or UNIQUE.

```
CREATE TABLE students (
    id INT PRIMARY KEY,
    name VARCHAR(100),
    age INT CHECK (age >= 18) -- Attribute constraint
);
```

- Tuple-Based Constraints: These apply to the entire row, often via triggers or custom checks.

```
CREATE TABLE bookings (
    start_time TIMESTAMP,
    end_time TIMESTAMP,
    CHECK (end_time > start_time) -- Tuple constraint
);
```

Aggregation Functions

Aggregation functions perform calculations on multiple rows to produce a single result. Common aggregation functions include SUM(), COUNT(), AVG(), MIN(), and MAX().

```
SELECT department_id, SUM(salary) AS total_salary
FROM employees
GROUP BY department_id;
```

This query calculates the total salary for each department.

GROUP BY

GROUP BY is used to group rows that share a property so that an aggregate function can be applied to each group.

```
sql
SELECT department_id, COUNT(*)
FROM employees
GROUP BY department_id;
```

This will group employees by department_id and count the number of employees in each department.

GROUP BY Multiple Parameters

You can group by multiple columns to create groups based on combinations of values.

```
SELECT department_id, job_id, COUNT(*)
FROM employees
GROUP BY department_id, job_id;
```

This will group by both department_id and job_id, counting the number of employees in each combination of department and job.

Joins

Joins combine rows from two or more tables based on a related column. The USING clause specifies the common column(s).

```
SELECT e.name, d.department_name
FROM employees e
JOIN departments d USING (department_id)
WHERE d.location = 'New York';
```

- This joins the employees and departments tables using department_id. The department_id column is used as the common column in both tables (employees and departments). The USING clause simplifies the syntax by allowing you to specify the common column without needing to repeat the table name.
- The where clause and departments in New York.
- The join used here is an ***inner join***. The USING clause specifies that the common column department_id exists in both the employees and departments tables. An inner join returns only the rows where there is a match in both tables.

Natural Join

A natural join is a type of join that automatically combines tables based on all columns with the same name and compatible data types in both tables. It eliminates duplicate columns in the result set.

```
SELECT *
FROM employees
NATURAL JOIN departments;
```

In this case, if both tables have a column named department_id, the natural join will automatically use that column to match rows from the two tables.

Join Using Specific Columns

When you specify a join condition using specific columns like e.name = d.department_name, it is referred to as **an inner join with an ON clause.**

```
SELECT e.name, d.department_name
FROM employees e
JOIN departments d ON e.name = d.department_name;
```

- In this case, the ON clause explicitly defines how the two tables should be joined based on the condition provided (in this case, the names matching).

ChatGPT can make mistakes. Check important info.

Normalization and Entity Relations

Normalization is the process of organizing data in a database to reduce redundancy and improve data integrity. The goal is to break down a table into smaller tables and define relationships between them to ensure the data is stored efficiently.

1st Normal Form (1NF):

Ensure that all columns contain atomic (indivisible) values. Each column should have unique data.

Non-normalized table:

```
| OrderID | CustomerName | ProductList         |
|---------|--------------|---------------------|
| 1       | John Doe     | Bread, Milk, Cheese |
```

- 1NF: Split into multiple rows:

```
| OrderID | CustomerName | Product   |
|---------|--------------|-----------|
| 1       | John Doe     | Bread     |
| 1       | John Doe     | Milk      |
| 1       | John Doe     | Cheese    |
```

2nd Normal Form (2NF):

Achieve 1NF and ensure no partial dependency exists. In 2NF, every non-primary key column must depend on the entire primary key.

Non-normalized table:

```
| OrderID | ProductID | ProductName | ProductPrice |
|---------|-----------|-------------|--------------|
| 1       | 101       | Bread       | 1.5          |
```

2NF: Split the table into two tables:

```
| OrderID | ProductID |
|---------|-----------|
| 1       | 101       |

| ProductID | ProductName | ProductPrice |
|-----------|-------------|--------------|
| 101       | Bread       | 1.5          |
```

3rd Normal Form (3NF):

Achieve 2NF and ensure that no transitive dependencies exist. In 3NF, non-primary key attributes should not depend on other non-primary key attributes.

| CustomerID | CustomerName | CustomerZipCode | CustomerCity | Break into two tables: lua

CustomerID	CustomerName	CustomerZipCode

ZipCode	City

Entity Relationships:

- One-to-One: One entity is related to only one entity.
- One-to-Many: One entity is related to many entities.
- Many-to-Many: Many entities are related to many other entities

Customer (1) -------- (M) Orders | | CustomerID OrderID Name ProductID Address Quantity

Entity Relation Diagram

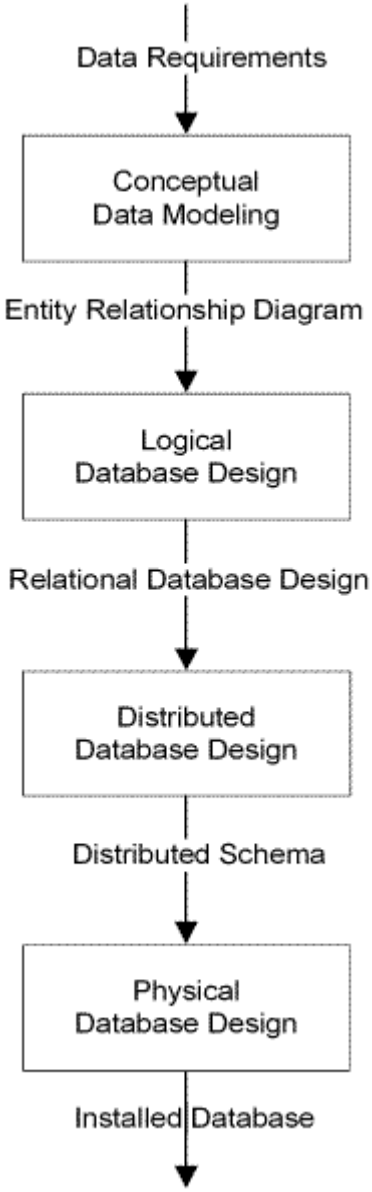
Entity Relationship (ER) Diagram is a type of flowchart, which shows how entities (people, objects) relate to each other within a system. Flowchart is a type of diagram that represents the process. ERD uses a defined set of symbols such as rectangles, diamonds, ovals and lines to depict interconnectedness of entities, relationships, and their attributes.

ER Diagrams are composed of entities, relationships, and attributes. And depict cardinality, which defines relationships in terms of numbers.

Normalizing with Entity Relationship Diagramming

Data modeling is performed during the initial phases of the database development process (also referred as database life cycle). During this process, the top two phases are concerned with the information content of

the database, while the last two phases are concerned with the implementation of the database on some commercial DBMS.



During the conceptual data modeling phase, data requirements are expressed through an ERD. The conceptual data modeling phase in general is independent of a DBMS. The logical design phase transforms the conceptual data model into a format understandable to DBMS. This phase may also enhance or refine the data model (ERD) of the previous phase to ensure efficient utilization of the database. Since most of the commercial DBMS are based on the relational model, the end product of this phase is relational model design.

One of the ways an ERD is enhanced during the logical design phase is through the process of normalization. Normalization is one of the key tenets in relational model design. It is the process of removing redundancy in a table so that the table is easier to modify. It usually involves dividing an entity table into two or more tables and defining relationships between the tables. The objective is to isolate data so that additions, deletions, and modifications of an attribute can be made in just one table and then propagated through the rest of the database via the defined relationships.

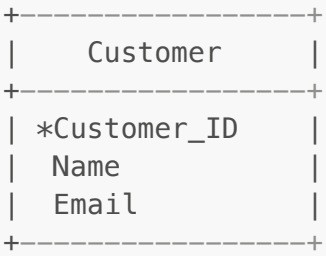
Normalization utilizes association among attributes within an entity table to accomplish its objective. Since an ERD also utilizes association among attributes as a basis to identify entity type structure, it is possible to

apply normalization principles during the conceptual data modeling phase. Performing normalization during ERD development can improve the conceptual model, and speed its implementation.

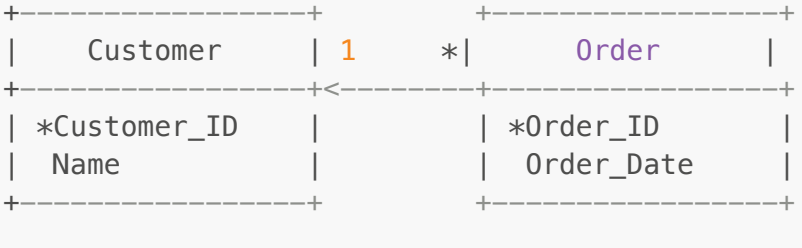
Entity Diagrams Vs Relationship Diagrams

Both used in database design, but they serve different purposes and focus on different aspects of the data model.

1. An Entity Diagram, often referred to as an Entity-Relationship Diagram (ERD), visually represents the entities (tables) in a database and their attributes (fields).
    - Components:
        - Entities: Represented as rectangles, these are the objects or concepts (e.g., Customer, Order, Product) for which data is stored.
        - Attributes: Represented as ovals or listed within the entity box, these are the data fields associated with the entities (e.g., Customer_ID, Name, Order_Date).
        - Primary Key: Underlined attributes that uniquely identify an entity.
    - Focus: It focuses on the individual entities and their attributes rather than the relationships between them.



2. A Relationship Diagram often refers to a more detailed representation of how entities relate to each other, typically part of an Entity-Relationship Diagram but emphasizing the relationships.
    - Components:
        - Relationships: Represented as diamonds or labeled lines connecting entities, indicating how entities are associated (e.g., one-to-many, many-to-many).
        - Cardinality: Indicates the nature of the relationship, specifying how many instances of one entity relate to instances of another (e.g., one customer can place many orders).
    - Focus: It emphasizes the connections and associations between entities rather than the entities themselves.



- Key Differences

    - Entity Diagram (ERD): Primarily shows entities and their attributes.
    - Relationship Diagram: Focuses on how entities are connected and the nature of these connections.
    - Entity Diagram: Utilizes rectangles for entities and ovals for attributes.
    - Relationship Diagram: Utilizes lines and diamonds to depict relationships and cardinalities.

T-SQL (Selects, Joins, Conditions)

T-SQL (Transact-SQL) is an extension of SQL used in Microsoft SQL Server. This means that it provides all the functionality of SQL but with some added extras.

- Put simply: when you are writing queries within Microsoft SQL Server, you are effectively using T-SQL. All applications that communicate with SQL Server, regardless of the application's user interface, do so by sending T-SQL statements to the server.
-
- SELECT: Used to retrieve data from tables.

```
SELECT FirstName, LastName
FROM Customers;
```

- JOIN: Used to combine rows from two or more tables.
- Types of Joins:
    - INNER JOIN: Returns only the matching rows between the tables.
    - LEFT JOIN: Returns all rows from the left table and matched rows from the right table.
    - RIGHT JOIN: Returns all rows from the right table and matched rows from the left table.
    - FULL JOIN: Returns all rows from both tables.

```
SELECT Orders.OrderID, Customers.CustomerName
FROM Orders
INNER JOIN Customers ON Orders.CustomerID = Customers.CustomerID;
```

- WHERE: Used to filter records.

```
SELECT *
FROM Customers
WHERE Country = 'USA' AND Age > 30;
```

- Conditions: Operators like AND, OR, and comparison operators (=, >, <, etc.) are used to apply conditions.

```

```

```
SELECT ProductName
FROM Products
WHERE Price BETWEEN 10 AND 50;
```

T-SQL vs SQL

- The obvious difference is in what they are designed for: SQL is a query language used for manipulating data stored in a database. T-SQL is also a query language, but it's an extension of SQL that is primarily used in Microsoft SQL Server databases and software.
- SQL is open-source. T-SQL is developed and owned by Microsoft.
- On top of these more general differences, SQL and T-SQL also have some slightly different command key words. T-SQL also features functions that are not part of regular SQL.
    - An example of this is how in we select the top X number of rows. In standard SQL, we would use the LIMIT keyword. In T-SQL, we use the TOP keyword.

```
--  SQL Example
SELECT *
FROM users
ORDER BY age
LIMIT 10;
--  T-SQL Example
SELECT TOP 10 (*)
FROM users
ORDER BY age;
```

Variables:

- Transact-SQL provides the following statements to declare and set local variables: DECLARE, SET and SELECT.

```
DECLARE @var1 NVARCHAR(30);
SET @var1 = 'Some Name';
SELECT @var1 = Name
  FROM Sales.Store
  WHERE CustomerID = 100;
```

Flow control

Keywords for flow control in Transact-SQL include BEGIN and `END`, `BREAK`, `CONTINUE`, `GOTO`, `IF` and `ELSE`, `RETURN`, `WAITFOR`, and `WHILE`.

`IF` and `ELSE` allow conditional execution. This batch statement will print "It is the weekend" if the current date is a weekend day, or "It is a weekday" if the current date is a weekday. (Note: This code assumes that

Sunday is configured as the first day of the week in the `@@DATEFIRST` setting.)

```
IF DATEPART(dw, GETDATE()) = 7 OR DATEPART(dw, GETDATE()) = 1
   PRINT 'It is the weekend.';
ELSE
   PRINT 'It is a weekday.';
```

`BEGIN` and END mark a block of statements. If more than one statement is to be controlled by the conditional in the example above, we can use BEGIN and END like this:

```
IF DATEPART(dw, GETDATE()) = 7 OR DATEPART(dw, GETDATE()) = 1
BEGIN
   PRINT 'It is the weekend.';
   PRINT 'Get some rest on the weekend!';
END;
ELSE
BEGIN
   PRINT 'It is a weekday.';
   PRINT 'Get to work on a weekday!';
END;
```

`WAITFOR` will wait for a given amount of time, or until a particular time of day. The statement can be used for delays or to block execution until the set time.

`RETURN` is used to immediately return from a stored procedure or function.

`BREAK` ends the enclosing WHILE loop, while CONTINUE causes the next iteration of the loop to execute. An example of a WHILE loop is given below.

```
DECLARE @i INT;
SET @i = 0;

WHILE @i < 5
BEGIN
   PRINT 'Hello world.';
   SET @i = @i + 1;
END;
```

Changes to DELETE and UPDATE statements

In Transact-SQL, both the `DELETE` and `UPDATE` statements are enhanced to enable data from another table to be used in the operation, without needing a subquery:

- `DELETE` accepts joined tables in the FROM clause, similarly to SELECT. When this is done, the name or alias of which table in the join is to be deleted from is placed between `DELETE` and `FROM`.

- UPDATE allows a FROM clause to be added. The table to be updated can be either joined in the FROM clause and referenced by alias, or referenced only at the start of the statement as per standard SQL.
- This example deletes all users who have been flagged in the user_flags table with the 'idle' flag.

```
DELETE u
FROM users AS u INNER JOIN user_flags AS f ON u.id = f.id
WHERE f.name = 'idle';
```

Indexes

Indexes are used to speed up searches and queries by allowing the database to find data without scanning the entire table.

- Clustered Index: Alters the physical order of the table and only one clustered index can exist per table.

```
CREATE CLUSTERED INDEX idx_CustomerID
ON Customers(CustomerID);
```

- Non-Clustered Index: Creates a separate structure to hold the index, and multiple non-clustered indexes can exist.

```
CREATE NONCLUSTERED INDEX idx_CustomerName
ON Customers(CustomerName);
```

```
Table (No Index)   |   Table (Indexed)
-----------------  |   -----------------
Scan entire table  |   Direct lookup
```

Types of Indexes

Keep in mind that the choice of index type depends on the specific requirements of your database, the nature of the data it holds, and the kind of operations you intend to perform on that data. Each index type has its own strengths and weaknesses, and understanding these can help you optimize your database's performance.

```
CREATE TABLE employees (
    employee_id INT PRIMARY KEY,
    first_name VARCHAR(50),
    last_name VARCHAR(50),
    email VARCHAR(100),
```

```
    department_id INT
);
```

1. Single-Column Indexes

Single-column indexes, as the name suggests, are created on a single column of a table. An index on a single column (e.g., `last_name`). It can improve the performance of queries that **filter** or **sort** by the last_name column.

```
CREATE INDEX idx_last_name ON employees(last_name);
```

2. Composite (Concatenated) Indexes

Composite indexes, also known as concatenated indexes, are built on two or more columns of a table. The composite index can improve the performance of queries that use both `last_name` and `first_name` in the `WHERE` clause, or when **sorting** by these columns.

```
CREATE INDEX idx_last_first_name ON employees(last_name, first_name);
---------------------------------------
SELECT *
FROM employees
WHERE last_name = 'Smith' AND first_name = 'John';
```

3. Unique Indexes

A unique index is a type of index that ensures all the values in the indexed column (or combination of columns, for composite unique indexes) are distinct. This means that no two rows in the table can have the same value(s) in the indexed column(s) If the database engine is asked to insert a duplicate entry into a unique indexed column, it will refuse the operation [just like a unique constraint].

```
CREATE UNIQUE INDEX idx_unique_email ON employees(email);


INSERT INTO employees (employee_id, first_name, last_name, email,
department_id)
VALUES (1, 'John', 'Doe', 'john.doe@example.com', 10);

-- Trying to insert another row with the same email
-- This will fail
INSERT INTO employees (employee_id, first_name, last_name, email,
department_id)
VALUES (2, 'Jane', 'Doe', 'john.doe@example.com', 20);  -
```

4. B-Tree Indexes

B-tree indexes are the most common type of index. They allow the database to find data by following a branching path, leading to the desired value. This makes them excellent for handling equality and range queries. B-tree indexes can be **applied to any data type that supports sorting** and can be used for indexing both single and multiple columns in a table.

```
-- By default, when you create an index in most databases, it will be a B-tree index
CREATE INDEX idx_employee_name ON employees(last_name);
```

5. Hash Indexes

Hash indexes use a hash function to map index keys to locations in a hash table. They are incredibly efficient for equality searches, but they are unsuitable for range queries or sorting operations. This is because the nature of a hash function is such that it scatters entries across the hash table, and there's no correlation between the proximity of keys in the index and their corresponding rows in the table.

- Structure: Hash indexes use a hash function to generate a unique hash code for each key value. This hash code is then used to locate the corresponding data in the table.
- Direct Mapping: When querying with an equality condition, the hash function maps the search key directly to the location of the data, which makes hash indexes extremely fast for such operations.
- Non-Sequential Storage: Unlike B-tree indexes, hash indexes do not store data in sorted order. They are designed for direct lookups rather than range-based searches.

```
CREATE INDEX idx_employee_id_hash ON employees USING hash (employee_id);
```

6. Clustered indexes

Clustered indexes organize the rows in a table or a view in a specific order to match the index. This order is dictated by the column or columns involved in the index. Therefore, the sequence of data in a clustered index aligns exactly with how the data is stored in the table, making it a highly efficient method for both range and individual record queries.

- It's important to note that there can only be one clustered index per table since the data rows themselves can only be sorted in one order. This does not, however, limit the number of non-clustered indexes that can be applied to the same table.

```
CREATE CLUSTERED INDEX idx_employee_id ON employees(employee_id);
```

7. Non-Clustered Indexes

Non-clustered indexes, on the other hand, have a structure separate from the data rows. This type of index contains the non-clustered index key values and each key value entry has a pointer to the data row that contains the key value. The pointer from an index row in a non-clustered index to a data row is called a **row**

locator. The structure of the row locator depends on whether the data pages are stored in a heap (A heap is a table without a clustered index or a non-clustered index) or a table with a clustered index.

```
CREATE NONCLUSTERED INDEX idx_last_name ON employees(last_name);
```

Heap

A heap is a table without a clustered index. In a heap, the data rows are not stored in any particular order. The rows are placed wherever there is space available on the data pages, leading to unsorted storage.

- In a heap, the data rows are not stored in any particular order. The rows are placed wherever there is space available on the data pages, leading to unsorted storage.
- Row Locator in a Heap: In a non-clustered index pointing to a heap, the row locator is a pointer to the **physical location of the data**. It usually consists of a **file identifier, page number, and row number** on the page where the data is stored.
- A heap is an unordered table where data is stored in the order it was inserted, making it suitable for fast inserts.
-

Table with a Clustered Index

A table with a clustered index has its data rows stored in **the order defined by the clustered index**. This means that the physical arrangement of the rows on the disk matches the order of the index.

- Row Locator in a Clustered Index: When a non-clustered index points to a table with a clustered index, the row locator is the **clustered index key value**. This is because the clustered index itself dictates the location of the data rows.
- A table with a clustered index has data physically stored in a sorted order, which improves the performance of queries that involve sorting or range conditions.

```
- In a non-clustered index, the row locator differs depending on
whether the data is stored in a heap (physical pointer) or in a table
with a clustered index (clustered index key).
```

8. Filtered Indexes

Filtered indexes are a type of non-clustered index with a WHERE clause, and they're **only available in Microsoft SQL Server**. They're used to index a subset of rows in a table, which can improve query performance, reduce index maintenance costs, and reduce index storage costs compared with full-table indexes.

```

```

```
CREATE INDEX idx_orders_pending ON orders(order_date)
WHERE status = 'Pending';
----------
SELECT *
FROM orders
WHERE status = 'Pending' AND order_date > '2024-01-01';
```

Stored Procedures (SPs)

- Predefined SQL queries saved in the database, which can be executed later to perform operations.
- So if you have an SQL query that you write over and over again, save it as a stored procedure, and then just call it to execute it.
- You can also pass parameters to a stored procedure, so that the stored procedure can act based on the parameter value(s) that is passed.

```
--- Stored Procedure Syntax
CREATE PROCEDURE procedure_name
AS
sql_statement
GO;

-- Execute a Stored Procedure
EXEC procedure_name;
```

- Setting up multiple parameters is very easy. Just list each parameter and the data type separated by a comma as shown below.

```
CREATE PROCEDURE SelectAllCustomers @City nvarchar(30), @PostalCode
nvarchar(10)
AS
SELECT * FROM Customers WHERE City = @City AND PostalCode = @PostalCode
GO;
--- execute
EXEC SelectAllCustomers @City = 'London', @PostalCode = 'WA1 1DP';
```

Views:

Views in a database are virtual tables created by a query that selects data from one or more underlying tables. Unlike materialized views, views do not store data themselves; they dynamically retrieve the data whenever they are accessed.

- Virtual Table: It behaves like a table when queried, but its data comes from the underlying base tables.

- Dynamic Data Retrieval: Since views don't store data, any changes to the underlying tables are reflected immediately in the view.
- Advantages:
  - Data Abstraction: Can be used to simplify complex queries or hide certain columns.
  - Security: Can restrict access to specific columns or rows by exposing only the necessary data.
  - Reusability: Allows for reusing common query logic across different applications.
- Limitations:
  - Performance: Since the data is not stored, querying large or complex views can be slower compared to querying materialized views.
  - Read-Only Limitations: Some views are not updatable, meaning you can't perform insert, update, or delete operations through the view.
- Use Cases:
  - To simplify complex joins and queries.
  - To present aggregated or derived data.
  - To control user access to sensitive information.
  -

```
Syntax
CREATE VIEW view_name AS
SELECT column1, column2.....
FROM table_name
WHERE condition;
-- Example
CREATE VIEW CustomerOrders AS
SELECT Customers.CustomerName, Orders.OrderID
FROM Customers
JOIN Orders ON Customers.CustomerID = Orders.CustomerID;
```

Modification to View Using Triggers

While views are typically read-only, you can make them updatable by using triggers.

```
CREATE TRIGGER update_salary
INSTEAD OF UPDATE ON employee_view
FOR EACH ROW
BEGIN
    UPDATE employees
    SET salary = :NEW.salary
    WHERE employee_id = :OLD.employee_id;
END;
```

This trigger allows the modification of the employee_view, which in turn updates the underlying employees table.

Materialized views

Materialized views are saved as physical tables in the database. Unlike regular views, which are stored as saved SQL queries and execute the query each time they are accessed, materialized views store the result set of the query as a table, which can be refreshed periodically or manually.

- They are not temporary tables; materialized views persist in the database and remain available until explicitly dropped or modified.

```
CREATE MATERIALIZED VIEW total_salaries AS
SELECT department_id, SUM(salary) AS total_salary
FROM employees
GROUP BY department_id;
```

- The results are stored and can be refreshed using:

```
REFRESH MATERIALIZED VIEW total_salaries;
```

Drawbacks of Updating Materialized Views:

- Performance Overhead: Refreshing a materialized view can be resource-intensive, especially if the underlying tables contain a large amount of data
- Staleness of Data: Materialized views can become stale(not-correct) quickly if the underlying data changes frequently. Depending on the refresh strategy (e.g., manual, incremental, or full), users may not see the most up-to-date data, which could lead to inconsistency in reporting or analysis.
- Complexity: Managing materialized views adds complexity to the database design. You need to handle the refresh logic and ensure that the materialized views are updated at the appropriate times, which may require additional programming or scheduling. Locking and Blocking: Depending on the database system and how the refresh is implemented, updating a materialized view may lock the underlying tables, potentially causing blocking issues for other transactions.

Views vs. Materialized Views

Aspect	Views	Materialized Views
Storage	No physical storage (virtual table)	Physically stores data
Data Freshness	Always current	Can be stale unless refreshed
Performance	Slower for complex queries, as data is computed	Faster, as data is precomputed and stored
Use Case	Simplifying queries, security layers	Performance optimization, data warehousing
Maintenance	No maintenance required	Requires refresh to stay in sync

Functions

Database functions are reusable pieces of code that perform a specific operation and return a value. Functions can be used within SQL statements, allowing for more dynamic and flexible queries. They can take input parameters, perform calculations or data transformations, and return results, which can be scalar values, table values, or other data types.

```
CREATE FUNCTION CalculateTotalPrice(@Price DECIMAL(10, 2), @TaxRate
DECIMAL(5, 2))
RETURNS DECIMAL(10, 2)
AS
BEGIN
    RETURN @Price + (@Price * @TaxRate);
END;

-- Usage
SELECT dbo.CalculateTotalPrice(100.00, 0.07) AS TotalPrice;
```

Table-Valued Functions

A function that returns a list of employees filtered by department.

```
CREATE FUNCTION GetEmployeesByDepartment(@DepartmentID INT)
RETURNS TABLE
AS
RETURN (
    SELECT EmployeeID, EmployeeName
    FROM Employees
    WHERE DepartmentID = @DepartmentID
);
--- Usage
SELECT * FROM GetEmployeesByDepartment(1);
```

Key Differences Between Views and Functions

Aspect	Views	Functions
Definition	Virtual table based on a query	Database object that returns a value or table
Usage	Used in `SELECT` statements like a table	Used in queries or as part of expressions
Return Type	Always returns a table	Can return scalar values, tables, or other data types
Data Storage	Does not store data, queries underlying tables	Does not store data, performs calculations dynamically

Aspect	Views	Functions
Procedural Logic	Not supported	Supports conditional logic (e.g., `IF`, loops)
Parameters	Cannot accept parameters	Can accept parameters
Complex Operations	Limited to query definitions	Can perform complex calculations and operations

Triggers

- A trigger is a stored procedure in a database that automatically invokes whenever a special event in the database occurs. For example, a trigger can be invoked when a row is inserted into a specified table or when specific table columns are updated.
- SQL code that automatically executes in response to certain events on a table or view (e.g., INSERT, UPDATE, DELETE).

```
create trigger [trigger_name]
[before | after]
{insert | update | delete}
on [table_name]
[for each row]
[trigger_body]
```

```
CREATE TRIGGER trg_AfterInsert ON Customers
FOR INSERT
AS
BEGIN
  PRINT 'A new customer has been added.';
END;
```

Different Trigger Types in SQL Server

- DDL Trigger
- DML Trigger
- Logon Triggers

DDL Triggers

The Data Definition Language (DDL) command events such as `Create_table`, `Create_view`, `drop_table`, `Drop_view`, and `Alter_table` cause the DDL triggers to be activated.

```
create trigger safety
on database
```

```
for
create_table,alter_table,drop_table
as
print 'you can not create,drop and alter tab
```

DML Triggers

The Data uses manipulation Language (DML) command events that begin with `Insert`, `Update`, and `Delete` set off the DML triggers. corresponding to `insert_table`, `update_view`, and `delete_table`.

```
create trigger deep
on emp
for
insert,update ,delete
as
print 'you can not insert,update and delete this table i'
rollback;
```

Logon Triggers

logon triggers are fires in response to a LOGON event. When a user session is created with a SQL Server instance after the authentication process of logging is finished but before establishing a user session, the LOGON event takes place.

Transactions

Transactions group a set of tasks into a single execution unit. Each transaction begins with a specific job and ends when all the tasks in the group successfully completed. If any of the tasks fail, the transaction fails. Therefore, a transaction has only two results: success or failure.

```
BEGIN TRANSACTION;

UPDATE Accounts
SET Balance = Balance - 100
WHERE AccountID = 1;

UPDATE Accounts
SET Balance = Balance + 100
WHERE AccountID = 2;

COMMIT TRANSACTION;
```

- This example ensures that the money transfer between two accounts is atomic (all changes happen or none at all).

Properties of Transaction - ACID

ACID stands for Atomicity, Consistency, Isolation, and Durability, which are key properties that ensure **reliable** transactions in a database system:

- **Atomicity**: Ensures that a transaction is all-or-nothing. If any part of the transaction fails, the entire transaction is rolled back, leaving the database unchanged.
- **Consistency**: Guarantees that a transaction brings the database from one valid state to another, maintaining all predefined rules (e.g., constraints [such as UNIQUE, FOREIGN KEY, etc], cascades).
    - **Cascade**: Involves the automatic propagation of changes in the database. For example, if a foreign key constraint with cascading delete is set up, deleting a parent record will automatically delete all associated child records to maintain referential integrity. Consistency ensures that such cascading operations are correctly applied to keep the database in a valid state.
    - Constraints ensure that all data follows specific rules.

```
CREATE TABLE students (
    id INT PRIMARY KEY,
    name VARCHAR(100),
    age INT CHECK (age >= 18)  -- Age must be 18 or higher
);
```

- This ensures that the age constraint holds in every transaction, maintaining database consistency.
- Constraints define rules for the data in a table. Types of constraints include PRIMARY KEY, FOREIGN KEY, CHECK, NOT NULL, and UNIQUE.

```


ALTER TABLE employees
ADD CONSTRAINT fk_department
FOREIGN KEY (department_id)
REFERENCES departments(department_id)
ON DELETE CASCADE;
```

- This enforces referential integrity, meaning if a department is deleted, the related employees are also deleted.
- **Isolation**: Ensures that transactions are executed independently without interference, even if they occur simultaneously.
- **Durability**: Guarantees that once a transaction is committed, it will remain permanent, even in the event of a system failure.

Isolation Levels Topics

Dirty Read

A dirty read occurs when a transaction reads data that has been modified by another transaction but hasn't been committed yet. If the other transaction rolls back, the data read may no longer be valid, leading to inconsistencies.

Non-Repeatable Read

A transaction reads data at one point and may be updated by another committed transaction before being read again. Thus, the data can change during the course of the same transaction.

Phantom Read

Phantom reads occur when, during a transaction, a query retrieves a set of rows that satisfies certain conditions, and in a subsequent execution of the same query within the same transaction, new rows (phantoms) have been added or deleted by another committed transaction, affecting the query results.

Phantom reads Vs non-repeatable reads

Non-repeatable reads involve changes to the data of existing rows, while phantom reads involve the appearance or disappearance of entire rows that affect the query result set.

Isolation Levels

Isolation levels define how transactions are isolated from each other. The most common levels are:

- Read Uncommitted:
    - Allows transactions to read data that has been modified by other uncommitted transactions.
    - Issues: May lead to **dirty reads**, where data could change or be rolled back.
- Read Committed (Default for most databases):
    - Only reads committed data from other transactions.
    - Prevents dirty reads, but allows "non-repeatable reads," where data read twice in the same transaction may differ.
    - Data read twice in the same transaction may still differ because other transactions can commit changes between the two reads. This situation is known as a **non-repeatable read**, where data retrieved at one point in a transaction may be updated by another committed transaction before being read again. Thus, the data can change during the course of the same transaction.
- Repeatable Read:
    - Ensures that if a transaction reads a row, subsequent reads within the same transaction will see the same data, even if other transactions update it.
    - Prevents dirty reads and non-repeatable reads but allows **phantom reads**, where new rows matching the query conditions can be added by other transactions.
- Serializable (Strictest level):
    - Transactions are executed in a way that they appear to be sequential, preventing dirty reads, non-repeatable reads, and phantom reads.
    - Achieves full isolation but may cause performance bottlenecks due to locking.

Comparison of Database Objects

Aspect	Functions	Stored Procedures (SPs)	Triggers	Transactions	Views
Definition	Returns a value, used in queries	Batch of SQL statements, can have complex logic	Executes automatically on specified events	A sequence of operations executed as a single unit	Virtual table based on a query
Return Type	Always returns a value	Can return values via output parameters or result sets	No return value, performs actions based on events	Does not return a value; controls data changes	Always returns a table
Input Parameters	Can accept parameters	Can accept input and output parameters	No input parameters; triggered by events	No parameters; operates on the context of the current transaction	Cannot accept parameters
Performance	Optimized for use in queries	Can be precompiled and cached	May impact performance if complex or frequent	Can improve performance by grouping operations	Slower for complex queries; no physical storage
Side Effects	No side effects	Can have side effects (e.g., modifying data)	Designed to perform side effects (e.g., enforcing integrity)	No side effects; ensures atomicity and consistency	No side effects; presents data without modification
Use Cases	Calculations, data transformation	Business logic, batch processing	Enforcing data integrity, logging	Ensuring data integrity and consistency	Simplifying queries, security layers
Maintenance	No maintenance required	Can require maintenance for logic changes	Requires maintenance if the underlying structure changes	Requires careful management to ensure consistency	No maintenance required

Aspect	Functions	Stored Procedures (SPs)	Triggers	Transactions	Views
Execution Context	Executes in the context of a query	Can execute in various contexts	Executes automatically based on data changes	Manages changes to data and ensures consistency	Executes in the context of the query

Temp Tables

Temporary Tables are most likely as Permanent Tables. Temporary Tables are Created in TempDB and are automatically deleted as soon as the last connection is terminated. Temporary Tables helps us to store and process intermediate results. Temporary tables are very useful when we need to store temporary data.

```
--- To Create Temporary Table:

CREATE TABLE #EmpDetails (id INT, name VARCHAR(25))
To Insert Values Into Temporary Table:

INSERT INTO #EmpDetails VALUES (01, 'Lalit'), (02, 'Atharva')
To Select Values from Temporary Table:

SELECT * FROM #EmpDetails

-- output
-----------
id    | name
-----------
1     | Lalit
2     | Atharva
```

- There are 2 types of Temporary Tables: Local Temporary Table, and Global Temporary Table.

Local Temporary Table:

A Local Temp Table is available only for the session that has created it. It is automatically dropped (deleted) when the connection that has created it, is closed. To create Local Temporary Table Single “#” is used as the prefix of a table name. Also, the user can drop this temporary table by using the `DROP TABLE #EmpDetails` query.

- There will be Random Numbers are appended to the Name of Table Name. If the Temporary Table is created inside the stored procedure, it get dropped automatically upon the completion of stored procedure execution.

```
CREATE PROCEDURE ProcTemp
AS
```

```
BEGIN
CREATE TABLE #EmpDetails
INSERT INTO #EmpDetails VALUES ( 01, 'Lalit'), ( 02, 'Atharva')
SELECT * FROM #EmpDetails
END
EXECUTE ProcTemp
```

Global Temporary Table

To create a Global Temporary Table, add the “##” symbol before the table name.

```
CREATE TABLE ##EmpDetails (id INT, name VARCHAR(25))
```

- Global Temporary Tables are visible to all connections and Dropped when the last connection referencing the table is closed.
- Global Table Name must have an Unique Table Name. There will be no random Numbers suffixed at the end of the Table Name.
- Use Case: Temporary tables are useful for breaking down complex queries or holding intermediate result sets in reporting.

CTEs (Common Table Expressions)

A CTE is a temporary result set defined within the execution scope of a single SELECT, INSERT, UPDATE, or DELETE statement. They help in writing cleaner and more readable queries by breaking down complex logic.

- We can define CTEs by adding a WITH clause directly before the SELECT, INSERT, UPDATE, DELETE, or MERGE statement.
- The WITH clause can include one or more CTEs separated by commas.

How to create a CTE

- Initiate a CTE using “WITH”
- Provide a name for the result soon-to-be defined query
- After assigning a name, follow with “AS”
- Specify column names (optional step)
- Define the query to produce the desired result set
- If multiple CTEs are required, initiate each subsequent expression with a comma and repeat steps 2-4.
- Reference the above-defined CTE(s) in a subsequent query

```
WITH
expression_name_1 AS
(CTE query definition 1)
```

```
[, expression_name_X AS
   (CTE query definition X)
 , etc ]

SELECT expression_A, expression_B, ...
FROM expression_name_1
```

```
WITH SalesCTE AS
(
    SELECT ProductID, SUM(Quantity) AS TotalQuantity
    FROM Sales
    GROUP BY ProductID
)
SELECT * FROM SalesCTE WHERE TotalQuantity > 100;
```

- joins need to use multiple nested subqueries which make it difficult to read or debug
- I might need to write teh same query multiple times, here it prevent me from writing it multiple times
- more readable and easy to maintain
- break big queries into multiple readable sections
- Use CTEs for recursive queries, simplifying hierarchical data retrieval or when breaking complex queries into more manageable parts.

Recursive CTEs and Hierarchical Data

Recursive CTEs enable you to process hierarchical data, such as a parts explosion (component, sub-components) or a management hierarchy (manager, employees).

```
WITH [ RECURSIVE ] <cte_name> AS
(
  <anchor_clause> UNION ALL <recursive_clause>
)
SELECT ... FROM ...;
```

anchor_clause

selects an initial row or set of rows that represent the top of the hierarchy. For example, if you are trying to display all the employees in a company, the anchor clause would select the President of the company.

```
SELECT employee_ID, manager_ID, title AS employee_title
             FROM employees
             WHERE manager_ID = null
```

- The anchor clause is the initial SELECT statement in a recursive CTE.

- The anchor clause cannot reference the `cte_name`.
- It can contain any valid SQL constructs, such as: Joins, Filters (e.g., WHERE conditions), Aggregations (GROUP BY, ORDER BY, LIMIT, etc.), Window functions (ROW_NUMBER(), RANK(), etc.)
- This clause sets the starting point for the recursion.

recursive_clause

- selects the next layer of the hierarchy based on the previous layer. In the first iteration, the previous layer is the result set from the anchor clause. In subsequent iterations, the previous layer is the most recent completed iteration.
- The recursive clause is the subsequent `SELECT` statement that references the `CTE itself`.
- It repeatedly executes and combines its results with the previous execution until a termination condition is met.
- The recursive_clause is a SELECT statement; however, the statement is restricted to **projections**, **joins**, and **filters**.
- projections refer to the process of selecting specific columns or computed values
- In addition, the following are not allowed in the statement:
  - Aggregate or window functions.
  - GROUP BY, ORDER BY, LIMIT, or DISTINCT.
- The recursive clause can reference the `cte_name` like a regular table or view.

```
SELECT  employees.employee_ID, employees.manager_ID, employees.title
               FROM employees JOIN cte_name
               ON employees.manager_ID = cte_name.employee_ID
```

This code recursively generates an organizational hierarchy by starting from the "President" and expanding downwards to other employees based on their manager relationships.

```
WITH RECURSIVE managers
    (employee_ID, manager_ID, employee_title)
AS
    (
        SELECT employee_ID, manager_ID, title AS employee_title
        FROM employees
        WHERE manager_ID = null

        UNION ALL

        SELECT employees.employee_ID, employees.manager_ID,
employees.title
        FROM employees JOIN managers
            ON employees.manager_ID = managers.employee_ID
    )

SELECT employee_title AS Title, employee_ID, manager_ID
FROM managers
```

Cursors

- Cursor is a Temporary Memory or Temporary Work Station.
- It is Allocated by Database Server at the Time of Performing DML(Data Manipulation Language) operations on the Table by the User.
- Cursors are used to store Database Tables.
- They are particularly useful when you need to perform operations on each row individually rather than applying a set-based operation to the entire result set. Here are some common use cases for cursors:
- There are 2 types of Cursors: Implicit Cursors, and Explicit Cursors. These are explained as following below.
    - Implicit Cursors: Implicit Cursors are also known as Default Cursors of SQL SERVER. These Cursors are allocated by SQL SERVER when the user performs DML operations.
    - Explicit Cursors: Explicit Cursors are Created by Users whenever the user requires them. Explicit Cursors are used for Fetching data from Table in Row-By-Row Manner.

1. This declares two integer variables, @ProductID and @Quantity, to hold the values fetched from the cursor.

```
DECLARE @ProductID INT, @Quantity INT;
```

1. In MySQL we can declare a cursor using the DECLARE statement and associate it with a SELECT statement to retrieve records from a database table.

```
-- This declares a cursor named SalesCursor that will iterate over the
ProductID and Quantity columns from the Sales table.
DECLARE SalesCursor CURSOR FOR
SELECT ProductID, Quantity FROM Sales;
```

1. After declaring a cursor in MySQL, the next step is to open the cursor using the OPEN statement. It initializes the result-set, allowing us to fetch and process rows from the associated SELECT statement in the cursor.

```
--  Open Cursor: This opens the cursor, making it ready to fetch data.
OPEN SalesCursor;
```

1. Then, we can use the FETCH statement to retrieve the current row pointed by the cursor, and with each FETCH, the cursor moves to the next row in the result set. This allows us to process each row one by one.

```
-- This fetches the first row from the cursor into the @ProductID and
@Quantity variables.
FETCH NEXT FROM SalesCursor INTO @ProductID, @Quantity;
```

1. Loop Through Rows:

```
WHILE @@FETCH_STATUS = 0
BEGIN
    -- Perform operation on each row
    UPDATE Inventory
    SET Quantity = Quantity - @Quantity
    WHERE ProductID = @ProductID;

    FETCH NEXT FROM SalesCursor INTO @ProductID, @Quantity;
END;
```

- This loop continues as long as @@FETCH_STATUS is 0 (indicating that the fetch was successful).
- Inside the loop, it updates the Inventory table by subtracting the fetched @Quantity from the current quantity for the corresponding @ProductID. Then, it fetches the next row.

6. Close and Deallocate Cursor:

```
CLOSE SalesCursor;
DEALLOCATE SalesCursor;
```

- After the loop finishes, the cursor is closed and deallocated to free up resources.

```
--- declare local variables used to fetch needed variables in loop
DECLARE @ProductID INT, @Quantity INT;
DECLARE SalesCursor CURSOR FOR
SELECT ProductID, Quantity FROM Sales;

OPEN SalesCursor;
FETCH NEXT FROM SalesCursor INTO @ProductID, @Quantity;

WHILE @@FETCH_STATUS = 0
BEGIN
    -- Perform operation on each row
    UPDATE Inventory
    SET Quantity = Quantity - @Quantity
    WHERE ProductID = @ProductID;

    FETCH NEXT FROM SalesCursor INTO @ProductID, @Quantity;
END;
```

```
CLOSE SalesCursor;
DEALLOCATE SalesCursor;
```

- Cursors are useful when you need to perform row-by-row operations,
- but they can be slow for large datasets, so set-based solutions are preferred when possible.
- Cursors in databases are used to retrieve, manipulate, and navigate through a result set row by row. They are particularly useful when you need to perform operations on each row individually rather than applying a set-based operation to the entire result set. Here are some common use cases for cursors:
- **Complex Operations**: When operations cannot be easily expressed in a single SQL statement and require procedural logic.
- **Iterative Calculations**: When calculations need to be performed iteratively across rows.
- **Dynamic SQL Execution**: When you need to execute dynamic SQL statements for each row in a result set.
- **Handling Large Result Sets**: When dealing with large result sets that cannot be processed efficiently in a single operation.

Dynamic Queries

In SQL Server, at times the SQL Queries need to be dynamic and not static, meaning the complete SQL query may be built dynamically at run time as a string using the user inputs and any specific application logic. This can be done in queries run from back-end applications or inside stored procedures.

- There could be scenarios where the Table Name, Column Name list, the 'WHERE' clause values, or any part of the query may be generated at run time from user inputs. So, this technique of dynamically constructing and executing SQL statements at run time based on user inputs which helps to create flexible and adoptable queries is referred to as Dynamic SQL in SQL Server.
- Building a dynamic SQL string can be determined by factors like string concatenation, parameter values, user input values, Stored procedure parameters, values received from calling a function or result from another subquery or any application logic.

```
DECLARE @SQLQuery AS NVARCHAR(1000);
DECLARE @TableName AS NVARCHAR(100);

SET @TableName = 'Sales';
SET @SQLQuery = 'SELECT * FROM ' + @TableName;

EXEC sp_executesql @SQLQuery;
```

- Once the dynamic SQL is built it can be run using the EXEC or sp_executesql commands

Dynamic SQL for Dynamic Filtering

Dynamic Filtering is a method in SQL Server dynamic query to filter data for different filter conditions dynamically for the same SQL query. The Filter condition could be an user input received from the front-end and passed to a dynamic query.

```
Create Procedure spFilterStudentData
@strStudentName nvarchar(100)
As
Begin
Declare @SqlQuery nvarchar(1000)
Set @SqlQuery= 'Select * from Students where Student_Name =' + CHAR(39) +
@strStudentName + CHAR(39)
Exec(@SqlQuery)
End
```

This stored procedure can be run using the below execution command:

```
EXEC spFilterStudentData 'Ramesh'
```

- Dynamic SQL is powerful for building flexible applications, such as search forms where users can specify multiple search criteria.
- However, it's important to prevent SQL injection by parameterizing queries properly.

Database Tunning

Database Performance Tuning refers to the process of optimizing database systems for improved performance and efficiency. It involves making adjustments in the database configuration, query design, indexing, and other system parameters to achieve maximum performance.

Techniques

Database Performance Tuning involves various techniques and methodologies aimed at improving database performance, such as:

- Query optimization: Rewriting SQL queries for better execution plans
- Index management: Creating, modifying, and deleting indexes to optimize data access
- Resource allocation: Assigning memory, CPU, and disk resources for optimal performance
- Database design: Designing database schemas/data pages that allow efficient data processing
- Data partitioning: Dividing large tables into smaller, more manageable pieces(Normalization) for improved query performance
- Caching: Storing frequently accessed data in memory for faster retrieval

How does Database Performance Tuning benefit businesses?

Benefits include increased query efficiency, reduced resource consumption(fewer computing resources), improved scalability (can handle more concurrent users and larger data volumes), better user experience, and higher return on investment(make the best use of hardware and software investments).

Simple Performance Tuning Techniques

Handling Indexes

Choosing the right indexes can be tricky. Indexes speed up read operations but can slow down write operations like INSERT or UPDATE.

- Index columns frequently used in WHERE, JOIN, and ORDER BY clauses.
- Avoid excessive indexing, as it can lead to performance issues with data modification queries.

Optimizing Aggregation

Aggregations like COUNT, SUM, AVG, etc., can be slow on large datasets.

- Use indexed columns in GROUP BY and ORDER BY.
- Consider materialized views or summary tables for pre-aggregated data.

Using the Right Data Types:

Choosing inappropriate data types can lead to inefficient storage and slower queries.

- Use the smallest data type that can hold the required data.
- Avoid using `VARCHAR` for fixed-length data; use CHAR instead. As `CHAR` storing data with a consistent length (Fixed-Length), while `VARCHAR` for data that may vary in length to save storage space.

Dealing with Complex Joins and Multiple Table Relationships:

Challenge: Joining multiple tables or handling complex relationships can slow down queries significantly.

- Reduce the number of tables being joined where possible.
- Use indexed columns for joining.

Balancing Normalization and Denormalization:

Highly normalized databases can result in many joins, while denormalized databases may have data duplication.

- Normalize data for write-heavy applications.
- Denormalize or use caching for read-heavy applications.

Auto-growth

- An auto-growth event is the process by which the SQL Server engine expands the size of a database file when it runs out of space.
- Each database has its own auto-growth setting – that is set by the default setting of the model database. Rarely do we use the model setting.
- The autogrowth number specifies the increment by which the file will grow each time it needs more space.

Why set up the auto-growth for each database?

- If you don't set up the auto-growth setting for each database and be proactive in your duties, then each time an auto-growth event is performed, SQL Server **holds up database processing** while an auto-growth event occurs. And this can cause poor performance.

What is the purpose of auto-growth and it's consequences of not setting up the auto-growth properly?

- Each time the auto-growth event occurs, the server need to expand this space and place the space in the disk.
- However, since the space allocated for the database may not be next to each other on the disk, the will cause your database file to become fragmented, thus poor performance.

How can I avoid auto-growth events from occurring too many times?

- Pre-Size your databases and size the auto-growth to an appropriate.

What is an index?

The principal purpose of an index is to speed up data retrieval. It does this by reducing (the less pages the better) the number of database data pages (8kb in size) that have to be visited/scanned. Similar to a phone book or a text book, each has an index to help retrieve the data fast by using the index rather than scanning the entire phone book or the text book. There are primarily two types of indexes: clustered indexes and non-clustered indexes. Let's look at an example of each.

- An index is a copy of selected columns of data from a table that can be searched very efficiently that also includes a low-level disk block address or direct link to the complete row of data it was copied from.

table scan (whole table) in execution plan if no index which is bad but with clusteresd index the execution plan would be Index seek And an index seek basically goes to the goes to, if you will, directly to the pages where it starts with M, And let's look for the next word A, And if it finds any of the A's or Mary or Mary's or whatever, the key word is, it'll jump to directly to those pages and retrieve the pages.

What is clustered index and non-clustered index?

- In SQL Server, a clustered index determines the physical order of data in a table. There can be only one clustered index per table
- A non-clustered index is a special type of index in which the logical order of the index does not match the physical stored order of the rows on disk.

```
--EXAMPLE OF A CLUSTERED INDEX:THE PHONE BOOK

CREATE TABLE Names
(NameID INT IDENTITY,
FN varchar (50),
LN varchar (50))
GO
-- FIND FIRST NAME : HOLLY
-- TO FIND THE NAME HOLLY, YOU WILL HAVE TO GO THROUGH EACH ROW ONE BY ONE
UNTIL YOU REACH HOLLY; AND EVEN WHEN YOU SEE THE FIRST NAME HOLLY,
-- YOU WILL HAVE TO CONTINUE UNTIL THE END OF THE COLUMN, BECAUSE THERE
CAN BE MORE THAN ONE HOLLY.  AS YOU CAN SEE IT IS VERY TIME CONSUMING
-- AND WILL CAUSE POOR PERFORMANCE.
select FN, LN
```

```
from Names
where FN = 'Mary'
-- Try view the execution plan
SET STATISTICS IO ON
--display information regarding the amount of disk activity generated by
Transact-SQL statements
--logical reads  -- Number of pages read from the data cache.
--physical reads  -- Number of pages read from disk.


select FN, LN
from Names
where FN = 'Mary'
-- The select query above show "table scan" in execution plan--<< this is
not good for performance
--logical reads 76

CREATE CLUSTERED INDEX idx_FN ON Names(fn)
GO
--Now create a clusteded index on FN and review the execution plan
--Now that we have created a clusteded index on FN lets review the
execution plan; and look at the message for logical reads
select FN, LN
from Names
where FN = 'mary'

--logical reads 3
```

```
Create table Orders
(
Orderid int primary key,  --<< when you use a primary key constraint, you
automatically create a clustered index!
Orders varchar (20),
Cost varchar (20))
```

WHAT IS A NON CLUSTERED INDEX

```
A non clustered index is a separate structure for the base table.
```

- As you can see we have a clustered index on ID column; we also know that we can't create another clustered index on another column, as it will violate clustered index logic – we can't have both an organized column on ID and FN

CLUSTERED INDEX ON ID

BASE TABLE: STUDENTS

ID	FN	LN	AGE
1	BOB	SMITH	25
2	TOM	JONES	32
3	MARY	SAUNDERS	23
4	JOE	KIRKLAND	23
5	SUE	HIGGINS	32

- This is where non clustered index play an important part; because you can have many non clustered indexes. This is because a non clustered index is SEPERATE STRUCTURE from the base table. As a result, disk space become an issues!
- As you can see, when creating a non clustered index on a column, we can create more than one index;
- that is because the non clustered index is a SEPARATE STRUCTURE from the base table (just like the back of books have a separate index).
- We have pointers **(row locators)** that map the data on the base table.

CLUSTERED INDEX ON ID

ID	FN	LN	AGE
1	BOB	SMITH	25
2	TOM	JONES	32
3	MARY	SAUNDERS	23
4	JOE	KIRKLAND	23
5	SUE	HIGGINS	32

NON CLUSTERED INDEX ON FN

FN	ROW LOCATOR
BOB	111
TOM	222
MARY	333
JOE	444
SUE	555

NON CLUSTERED INDEX ON LN

LN	ROW LOCATOR
SMITH	111
JONES	222
SAUNDERS	333
KIRKLAND	444
HIGGINS	555

EXTRA SPACE IS NEEDED FOR THESE TWO NON CLUSTERED INDEXES

- This improves the performance of frequently used queries which are not covered by the clustered index –
- Note that when creating non clustered indexes on columns, disk space become an issues!
- If you have a million rows of FN and you create a non clustered index on FN, then you will also have a separate structure of a million rows with the data, thereby needing more disk space!!

B-tree

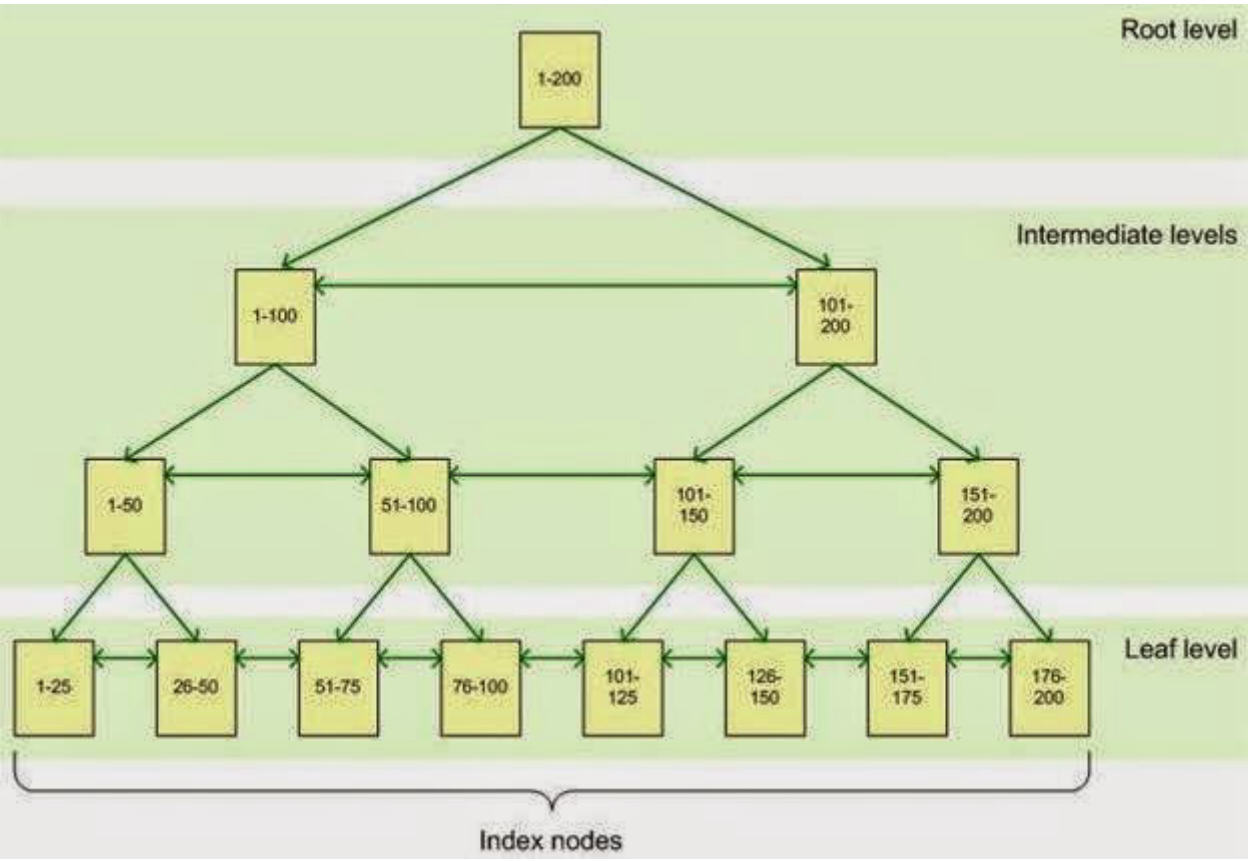
```
 When we create an index, whether clustered or non-clustered, the index is spread across a set of pages,
  referred to as the index nodes. We call this structure the B-tree structure.
   The pages are spread across three levels: The root node, the intermediate node, and the leaf nodes.
```

- A B-tree index orders rows according to their key values (remember the key is the column or columns you are interested in), and divides this ordered list into ranges. These ranges are organized in a tree structure, with the root node defining a broad set of ranges, and each level below defining a progressively narrower range. So how does this B tree structure help performance?

Suppose you have a query that is:

```
select orderid, orders
from Orders
where orderid = 76
```

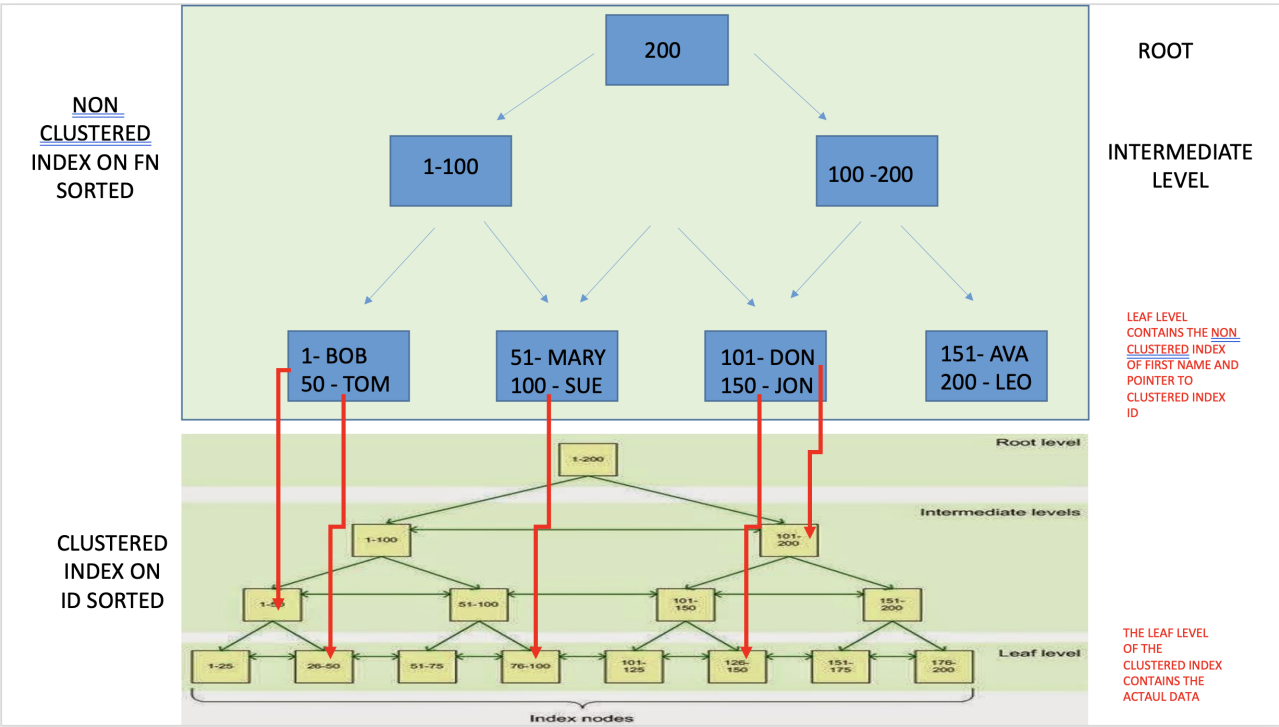
When we execute this query, the root node represents the main entry point for queries trying to locate data via the index. From that root node, the data engine negotiates the hierarchy down to the appropriate leaf node, where the actual data resides.



- The query engine starts at the root node, and determines if the orderid is between 1-100 or between 101-200. Since going to the right pointer is false, it will navigate down the left path. Then it ask again, if the ordered is between 1-50 or between 51-100. Since going to the left is false, it will navigate to the right.
  It then ask if the ordered is between 51-75 or between 76-100.
  Since going to the left is false, it will navigate to the right. From here it will go to the 76-100 leaf node and return the data. As this is a clustered index, the leaf node will contain the entire row of data associated with the key value 76.

- If this is a non-clustered index, the leaf node **will point to the clustered table or heap** where the row exists.

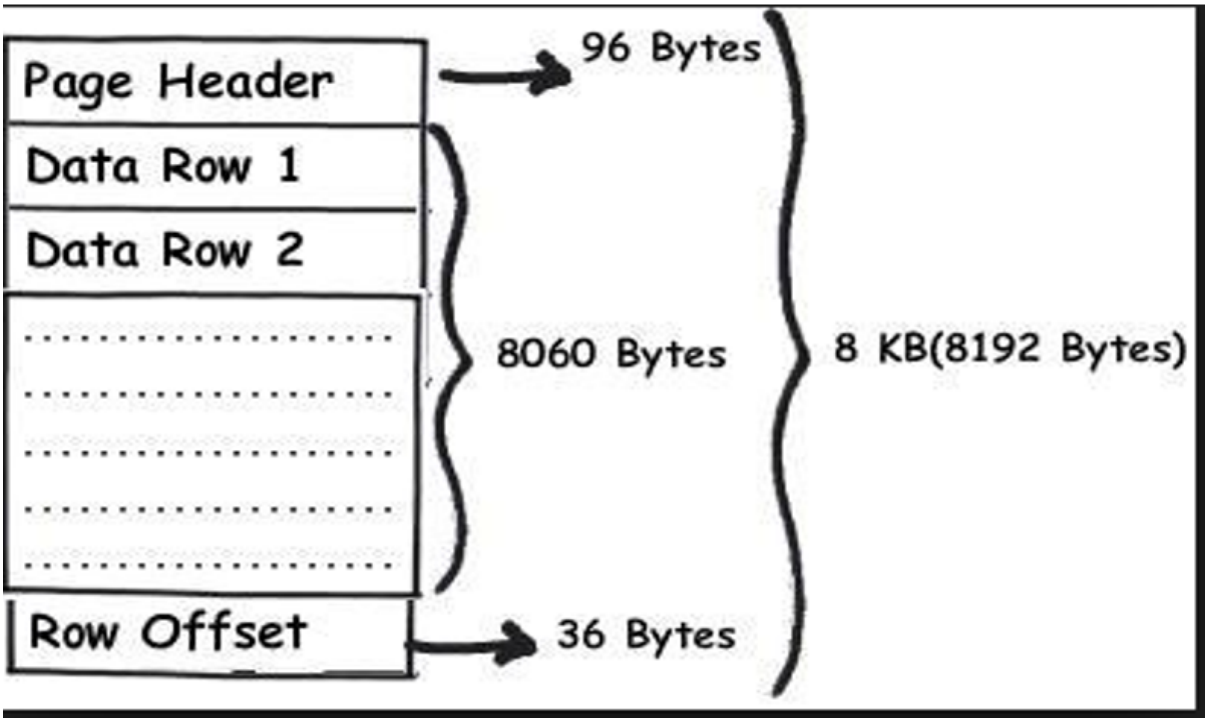
- As you can see, we can navigate to the data much faster and read less pages that a full table scan



SQL Page

A SQL page is the fundamental unit of data storage in SQL Server. It's **8KB(8000 bytes)** in size.

- A SQL page consist of 8KB (8192 bytes)
- The SQL page consists of 3 sections
    - Page header: 96 bytes
    - Row(s) of data: 8060 bytes per page
    - Row offset: 36 bytes



- If you design data rows that can fit into a single page then they'll take up a lot less storage (a lot less storage and less scanning of pages make for better performance)
- If for example, your have a row of 4200 bytes long then only a single row will be stored on a page (and the rest of the page - 3860 bytes - is wasted space).
- Thus, the important point is you want to try to avoid large areas of space that cannot hold any useful information on a page.
- If you redesigned the row size to 4000 bytes, then suddenly you can store two rows on a page, and thus significantly reduce the overhead of wasted space.
- This management of page size also play and important role in managing page splitting which can cause performance issues

Types of SQL pages

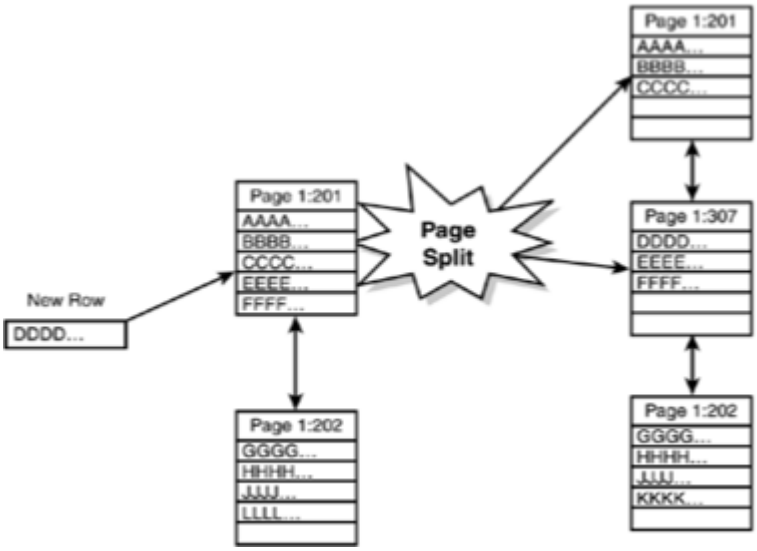
- There are many types of SQL pages.
- Focus on data page, index page and IAM page. Note the page type. 1 = data page, 2 = index page and 10 = IAM page.

Page Type	Page Name	Description
1	Data Page	Details about how the data is actually stored inside the database data files
		Clustered index leaf level information.
2	Index Page	Use for Indexing purpose.
		Non clustered index leaf information.
3	Text Mixed Page	Used for small LOB values for multiple rows and for multiple types.
4	Text Page	Used for LOB value for single column.
7	Sort Page	Used for Sort operations on temporary pages.
		Mostly used in tempdb database.
8	GAM Page	Used to track allocation of extent.
9	SGAM Page	Used to track allocation of shared extent.
10	IAM Page	Contains page information, memory information.
11	PFS Page	Contains information of free space of pages.
13	Boot Page	Contain information about the page.
14	Server Configuration Page	Contains information returned from sp_configure.
15	File Header Page	Contains information about the file.
16	Differential Changed map	Contains the extents information in GAM interval that have changed since last full or differential backup.
17	Bulk Change Map	Contains the extents information in GAM interval that have modified by bulk operations since last backup.

Page Splitting

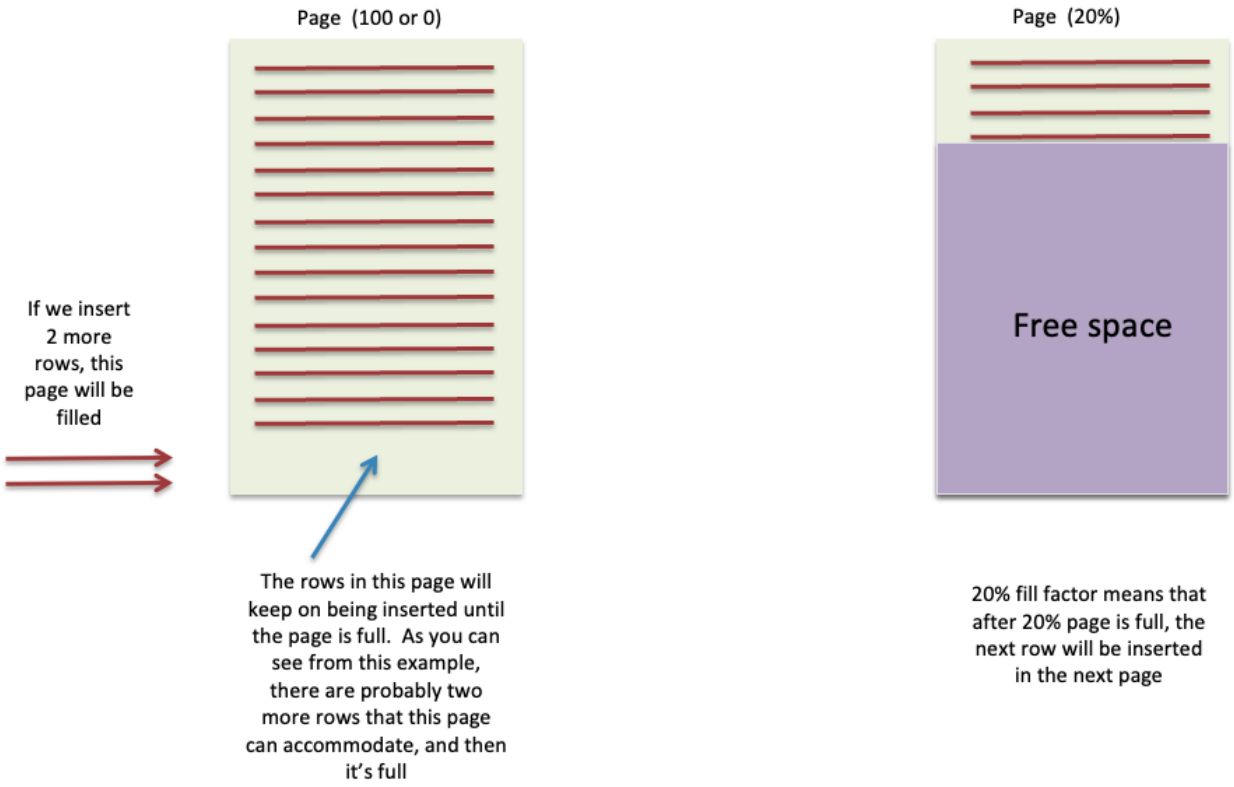
- At times, when you insert more rows there may not enough room for the newly inserted row in the applicable data page.
- If this happens, then SQL Server will have to move some of the data from the current data page and move it to a new data page.
- This involves the normal I/O (I/O -> migration to disk) of inserting of a row, but at the same time involves updating any applicable indexes.
- This result in a lot of excessive I/O.

- While a certain amount of page splits is normal and expected, too many page splits can cause performance issues.
- We can see that page `1:201` has five rows AAAA to FFFF not including DDDD
- When you insert a new row (DDDD) in a sequential order for the page that has NO MORE room for row DDDD, the SQL Server splits the page `1:201` in half and creates a NEW page `1:307` IN BETWEEN `1:201` and page `1:202`
- SQL server now has to read more pages and update the index also!



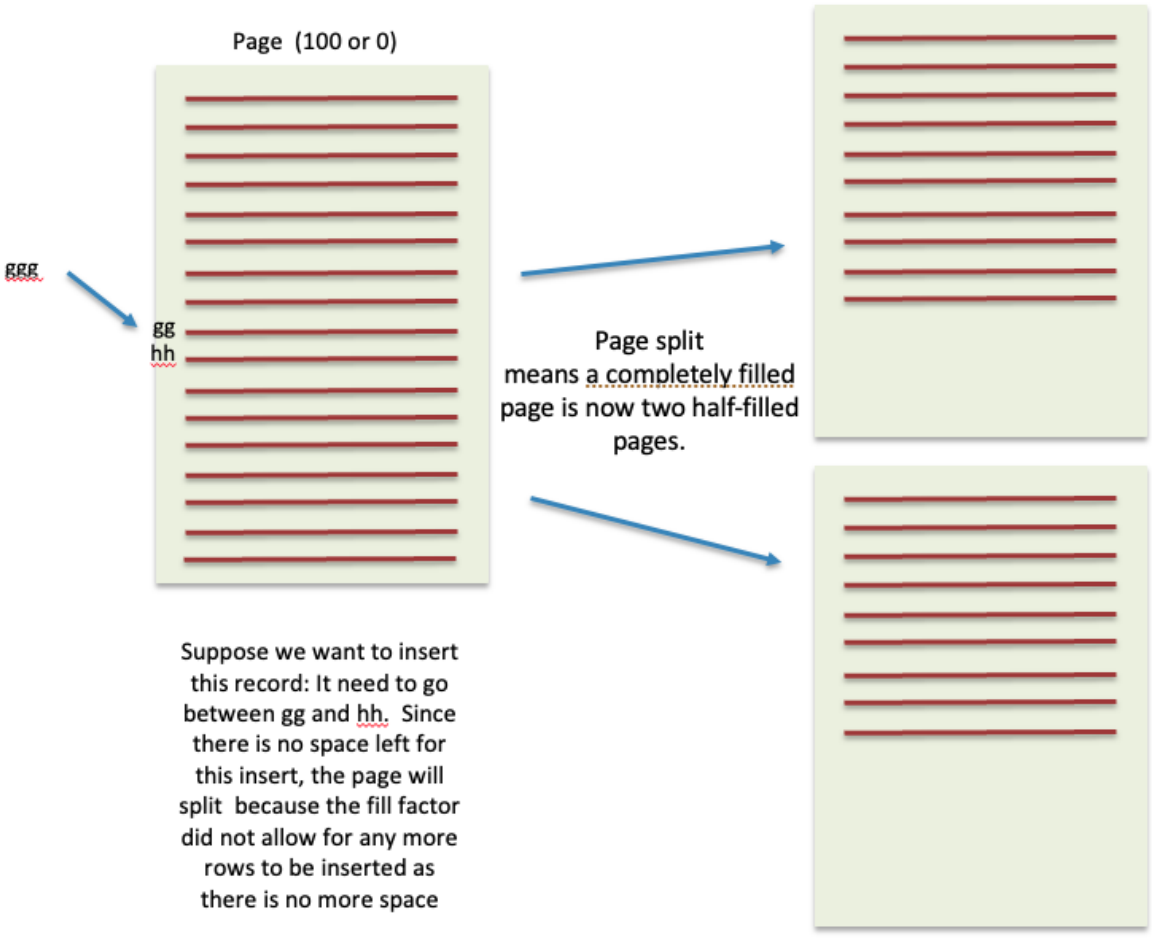
Fill Factor

- We know that each database page (8kb in size) stores one or more rows based on the size of the row. As such, the larger the row size, the fewer rows each page can have.
- The fill-factor value determines the percentage of space on each leaf-level page that can be filled with data, reserving the remainder on each page as free space for future growth.
- For example, specifying a fill-factor value of 80 means 80% of the page will have data and the remaining 20% of each leaf-level page will be left empty(on purpose for DML transactions), providing space for index expansion as data is added to the underlying table. Default of fill factor is (100 or 0).



What is the significance of understanding the fill factor?

The significance is this: If the page is completely filled and new data is inserted in the table which belongs to completely filled page, the "page split" event happens to accommodate new data.



- The reason why we need to keep an eye on fill factor: I/O
- If we had 5000 pages that had the data for a specific index or table and we split the pages in half to accommodate the newly inserted data, the SQL engine now has to read more pages to get the same amount of data, thereby costing a greater I/O consumption.

These added pages and higher I/O can cause performance issues

If we know that a fill factor of 100 can cause page splits, then what is the optimum number we should set for fill factor?

- Depends on your environment.
- If you have a read only database then set the fill factor to 100
- If you have a database with little modification for inserts or updates then set the fill factor to 100
- If your system experiences high OLTP transactions, then consider lowering the value to 85 to 95
- This number will have to tested

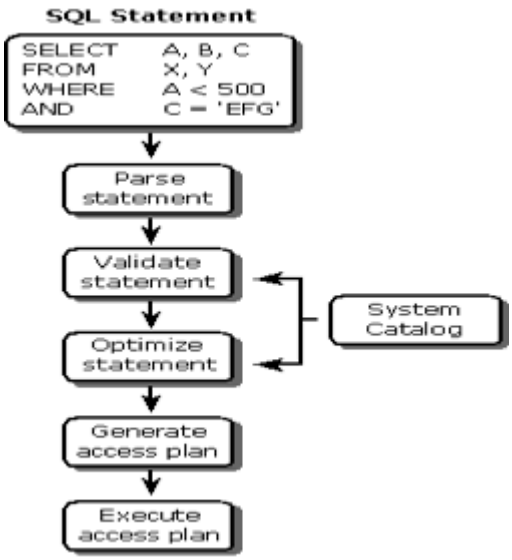
Query Optimizer

- When you submit a query to SQL server no matter how simple or complex, SQL Server generates an **execution plan** for that query.
- The execution plan shows you how SQL Server breaks down each query submitted by showing you a graphical representation using **operators**.
- There are 3 ways that an execution plan can be viewed:
    1. If you select the Actual Execution Plan option, then the plan is created after execution of the query
    2. If you select the Estimated Execution Plan then the plan is created without executing the query
    3. If you select the Live Query Statistics (SQL 2016 only) then you can view a live the execution plan of an active query without waiting for it to complete
- You can view the Execution plans can be presented in three ways:
    - Graphical Plans
    - Text Plans
    - XML Plans

How does the execution plan select the most efficient plan to execute?

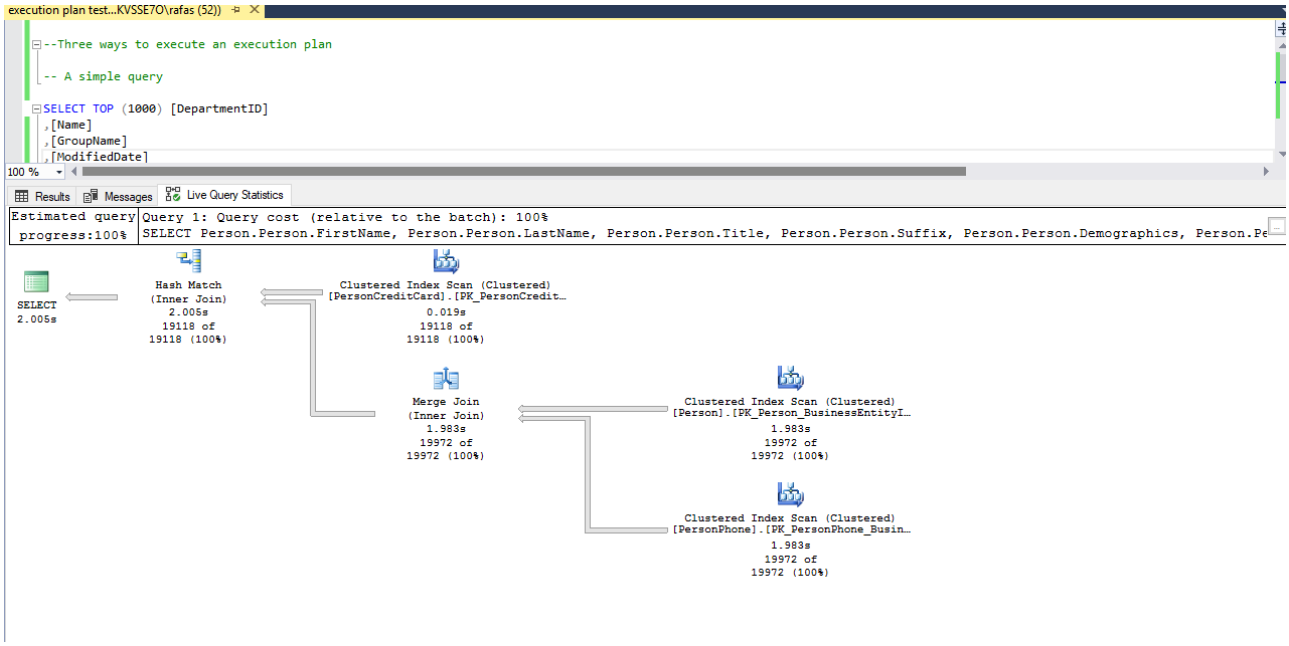
It use the query optimizer!

- **A query optimizer is part of the SQL Server engine that (SQL) queries and determines the most efficient execution plan**.
- It does this by optimizing each SQL statement based on statistics collected about the accessed data.
- The query optimizer then chooses the most efficient means of executing a SQL statement. The following diagram shows the steps the query optimizer uses create the execution plan:



Reading the Execution Plan

- You can read the Execution Plan (EP) either from right to left OR left to right
- When reading from left to right – you are asking what are the commands to get the data
- When reading from right to left– you are asking what is the flow of the data
- The thickness of the arrows shows the count of rows



Tool tip and its meaning

The tool tip when you hover over it gives you additional information about each operator. Each operator has its own tooltip and meaning.

Table scan (Heap) Vs Clustered Index Scan

```
--THIS SELECT STATEMENT PRODUCES A TABLE SCAN, BECAUSE THE THERE IS NO
INDEX.  AS SUCH IT RETRIVES THE DATA BY SCANNING THE ENTIRE
--TABLE GOING FORM ONE ROW TO ANOTHER  -- POOR PERFORAMCNE!  HOW CAN WE
FIX THIS?  CREATE A CLUSTERED INDEX! BUT FROM THIS COLUMN LIST
--WHICH COLUMN SHOULD WE SELECT FOR BEST PERFORMANCE?
```

```
SELECT * FROM [AdventureWorks2016CTP3].[Person].[PersonA]

CREATE CLUSTERED INDEX [ClusteredIndex-FN]
ON [Person].[PersonA]
GO

/*

AFTER CREATING AN INDEX ON FN, THE QUERY IS USING THE CLUSTERED INDEX - NO
TABLE SCAN. THIS IS BETTER, BUT IT STILL IS SCANNING THE ENTIRE DATA
AS THERE IS NO WHERE CLAUSE TO FILTER DATA

*/

SELECT * FROM [AdventureWorks2016CTP3].[Person].[PersonA]
--logical reads 3799
-- but the execution plan show CLUSTERED SCAN
```

A CLUSTERED INDEX

- Clustered indexes sort and store the data on their key values – as such only one clustered index
- Generally, every table should have a clustered index defined on the column
- Create clustered index on frequently used queries and high degree of uniqueness.
- That return large result sets
- Use JOIN clauses, ORDER BY, or GROUP BY clauses

NON CLUSTERED INDEX GUIDE

- Nonclustered indexes should be designed for frequently used queries
- Nonclustered indexes are optimal choice for EXACT match queries because the index contains entries describing the exact location in the table
- Those tables with low update requirements -databases that contain heavily updated tables should avoid over-indexing.
- Read-only data can benefit from many nonclustered indexes OLTP applications and large numbers of indexes on a table affect the performance of INSERT, UPDATE, DELETE, and MERGE statements BECAUSE all indexes must be adjusted appropriately as data in the table changes Use JOIN or GROUP BY clauses Create multiple nonclustered indexes on columns involved in join and grouping operations Create a non cluster a column(s) frequently involved in search conditions of a query using WHERE clause Create a non cluster a column(s) to cover the query (meaning if the where clause has more than one predicate, cover those columns) When the query optimizer can locate all the column values within the index; table or clustered index data is not accessed resulting in fewer disk I/O operations.

```
-- NOW USING THE WHERE CLAUSE FOR CITY, READS 341 AND STILL USING
CLUSTERED INDEX on id
select [City],[AddressLine1],[StateProvinceID],[PostalCode]
FROM [AdventureWorks2016CTP3].[Person].[AddressA]
```

```
where city = 'Oakland'
--logical reads 341

--CREATING A NON CLUSTERED INDEX ON CITY

USE [AdventureWorks2016CTP3]
GO

CREATE NONCLUSTERED INDEX [NonClusteredIndex-CITY]
ON [Person].[AddressA]
([City] ASC)

--AFTER CREATING A NON CLUSTERED INDEX ON COLUMN CITY AND USING THE SAME
QUERY WITH A SINGLE COLUMN CITY, READS = 3, AND USING THE NON CLUSTERED
INDEX

select [City]
FROM [AdventureWorks2016CTP3].[Person].[AddressA]
where city = 'Oakland'
--logical reads 3
```

COVERED INDEX

--A covering index is an index that contains all of the columns you need for your query

```
--HOWEVER, WHEN USING MORE THAN ONE COLUMN IN THE SELECT STATEMENT, WE GET
193 READS AND A KEY LOOKUP AGAINST THE CLUSTERED INDEX
--AND COST IS 99% AND A NESTED LOOP OPERATOR.  WHY?  BECAUSE THE WE ADDED
COLUMN ([AddressLine1]) WHICH IS NOT PART OF THE NON CLUSTERED INDEX
--AND THUS IT REACHES OUR TO THE CLUSTEDED INDEX TO RETRIVE THAT DATA AND
IT DOES A TABLE SCAN.  EXPENSIVE!
--HOW CAN WE GET RID OF THAT?  CREATE A COVERED INDEX.

select [City],[AddressLine1]
FROM [AdventureWorks2016CTP3].[Person].[AddressA]
where city = 'Oakland'
--logical reads 193

--CREATING A COVERED INDEX ON BOTH COLUMN IN THE SELECT STATEMENT

USE [AdventureWorks2016CTP3]
GO

CREATE NONCLUSTERED INDEX [NonClusteredIndex-COVER]
ON [Person].[AddressA]
([AddressLine1] ASC,[City] ASC)  --<< NOTE THAT THE NEW NON CLUSTERED
INDEX HAS BOTH COLUMNS

--AFTER COVERED INDEX - EXECUTING THE SAME QUERY - THE KEY LOOKUP IS GONE
BECUASE THE DATA IS DIRICTLY GOING TO THE NON CLUSTERED INDEX
```

```
select [City],[AddressLine1]
FROM [AdventureWorks2016CTP3].[Person].[AddressA]
where city = 'Oakland'
--logical reads 171

--  WE STILL GOT A NON CLUSTERED INDEX SCAN AND NOT A SEEK.
--WHY?  THAT IS BECAUSE THE ORDER OF THE COLUMNS WAS NOT IN THE RIGHT
ORDER, AS THE INDEX HAD THE ADDRESSLINE1 FIRST AND THEN THE CITY.
--IF WE REVERSE THE COLUMN NAME WE WILL GET A SEEK!!

select [City],[AddressLine1]
FROM [AdventureWorks2016CTP3].[Person].[AddressA]
where city = 'Oakland'
--logical reads 171

CREATE NONCLUSTERED INDEX [NonClusteredIndex-COVER]
ON [Person].[AddressA]
([City] ASC,[AddressLine1] ASC)  --<< NOTE THAT THE NEW NON CLUSTERED
INDEX HAS BOTH COLUMNS BUT I HAVE SWITCHED THE ORDER OF THE COLUMNS

--SAME QUERY BUT THIS TIME WE HAVE A SEEK!!

select [City],[AddressLine1]
FROM [AdventureWorks2016CTP3].[Person].[AddressA]
where city = 'Oakland'
--logical reads 171
```

- Once you add `AddressLine1` to the `SELECT` clause, the non-clustered index on City alone does not contain the AddressLine1 data. So, the query engine performs a key lookup to the clustered index to get the AddressLine1 values. This adds overhead.
- A key lookup occurs when the query optimizer needs to look up additional columns not included in the non-clustered index. It fetches rows from the clustered index for each matching row found in the non-clustered index, which can be expensive for large datasets.
- The key lookup involves more I/O operations, significantly increasing the execution cost.

OLAP (Online Analytical Processing) and OLTP (Online Transaction Processing) in databases:

OLAP:

Supports complex analysis and decision-making.

- Data Type: Historical data, often aggregated.
- Operations: Complex queries involving data aggregation (e.g., summing, averaging).
- Use Case: Business intelligence, data mining, and reporting.

OLTP:

Supports day-to-day transactional operations.

- Data Type: Current data, frequently changing.
- Operations: Simple queries involving insert, update, delete.

- Performance Focus: Optimized for write-heavy operations.
- Use Case: Inventory management, order processing, customer transactions.

Filtered Indexes

We have discussed that when creating a non-clustered index, we have to take in account the extra space needed for each of the non-clustered index we create. But the benefits of traditional indexes as we have seen come at a cost. One of the solutions is to use filtered indexes. We simply have to put a WHERE clause when creating a non clustered filtered. This way only the selected records are indexed providing a smaller utilization of disk space.

- Some of the benefits of using the filtered index are as follows:
    - Reduced storage cost – will need less disk for non clustered indexes
    - Optimized query performance - less data means the optimizer does not have to use as much statistics
    - Reduced index maintenance costs – the rebuilding and reorganizing of indexes is reduced

```
CREATE NONCLUSTERED INDEX [NonClusteredIndex-FILTER_CITY]
ON [Person].[AddressB]
([City] ASC) WHERE CITY = 'Seattle'   --<< THE WAY TO CREATE A FILTER NON
CLUSTERDE INDEX IS TO ADD THE WHERE CLAUSE

--SchemaName    TableName   RowCounts   TotalSpaceKB    UsedSpaceKB
UnusedSpaceKB   IndexType
--  Person  AddressB     141           72              16          56
NONCLUSTERED     --<< FILTER
--  Person  AddressB     19614        2896            2744         152
CLUSTERED
--  Person  AddressB     19614         712             632         80
NONCLUSTERED

SELECT CITY
FROM [AdventureWorks2016CTP3].[Person].[AddressB]
WHERE CITY = 'SEATTLE'
```

The SORT operator

The SORT operator sorts all rows received by the operator into order.

- When you view the SORT operator, it can be advisable to get rid for that operator in the execution plan as it can be expensive both in memory and on disk and can cause performance issues.
- The easiest way to avoid a SORT is by creating an Index.
- As we know, indexes are ordered by the columns so that , if you create an index covering your query, the Query Optimizer identifies this index and uses it to avoid a SORT operation.

```
--SORT OPERATOR: NOTICE THAT 82% OF THE COST IS TAKEN BY THE SORT COMMAND
(in execution plan)
```

```
SELECT City,AddressLine1
FROM [AdventureWorks2016CTP3].[Person].[AddressB]
ORDER BY [City]  --<, WE ARE USING THE ORDER BY COMMAND (SORTING)


--SIMPLEST WAY TO AVOID A SORT OPERATOR IS TO CREATE A CLUSTERED INDEX

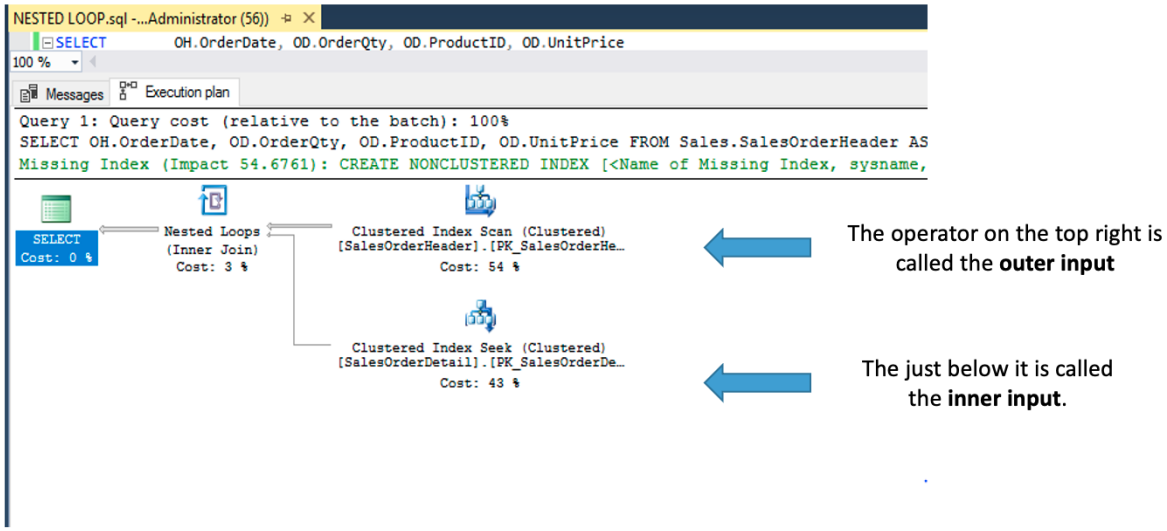
CREATE CLUSTERED INDEX [ClusteredIndex-20171026-165423]
ON [Person].[AddressB]
([City]ASC)


--NOW LET REVIEW THE EXUECTION PLAN.  NOTICE THAT THERE IS NOT NEED TO
SORT THE DATA AND THE SORT OPERATOR IS AVOIDED

SELECT City,AddressLine1
FROM [AdventureWorks2016CTP3].[Person].[AddressB]
ORDER BY [City]  --<, WE ARE USING THE ORDER BY COMMAND (SORTING)
```

Joins in execution plan

- Nested loop join (indicate that both table in join has an index)
- Hash Join (indicate that both table are heap - no indexes on either)
- Merge join (indicate that both table have clustered index on column used to join with) What is a nested loop join? Hash Join? Merge join?




```
--Example of a NESTED loop join
--Where the smaller table does not have an index BASED on the column being
used in an index, but the larger does have an index

SELECT * FROM  Sales.SalesOrderHeader AS OH
WHERE OH.OrderDate BETWEEN '2011-07-01' AND '2011-07-14'  --<< SMALLER
```

```
TABLE (144 ROWS) (OUTER TABLE) DOES NOT HAVE AN INDEX ON COLUMN ORDERDATE

SELECT * FROM  Sales.SalesOrderDetail  AS OD              --<< LARGER
TABLE (121317 ROWS) (INNER TABLE)


SELECT
OH.OrderDate, OD.OrderQty
FROM  Sales.SalesOrderHeader AS OH
INNER JOIN
Sales.SalesOrderDetail AS OD
ON OH.SalesOrderID = OD.SalesOrderID
WHERE (OH.OrderDate BETWEEN '2011-07-01' AND '2011-07-14')  --<< SMALLER
TABLE (144 ROWS) (OUTER TABLE) DOES NOT HAVE AN INDEX ON COLUMN ORDERDATE

--Example of a HASH join
--This happens when the tables are not properly sorted, and/or there are
no indexes.
--When SQL Server Optimizer chooses the Hash join type, it's usually means
an index is missing

--CREATE 2 LARGE TABLE WITHOUT INDEXES. AS THERE ARE NO INDEX ON OR SORTS
ON THE TABLES
--THIS WILL CAUSE A HASH JOIN TO BE SELECTED BY THE OPTIMIZER

--USE AdventureWorks2016CTP3
--GO

--DROP TABLE TABLE1
--DROP TABLE TABLE2

CREATE TABLE  TABLE1
(id INT identity ,EVENTS varchar(60))

--INSERT SOME DATA

declare @i int
set @i=0
while (@i<100)
begin
insert into TABLE1 (EVENTS)
select EVENT from AdventureWorks2016CTP3.dbo.DatabaseLog
set @i=@i+1
end

--VIEW LARGE TABLE

SELECT * FROM TABLE1  --<<17900 ROWS

--CREATE A SECOND TABLE

CREATE TABLE  TABLE2
(id INT identity ,EVENTS varchar(60))
```

```
--INSERT SOME DATA

declare @i int
set @i=0
while (@i<100)
begin
insert into TABLE2 (EVENTS)
select EVENT from AdventureWorks2016CTP3.dbo.DatabaseLog
set @i=@i+1
end

--VIEW LARGE TABLE

SELECT * FROM TABLE2  --<<17900 ROWS


--EXECUTE AND VIEW HASH JOIN AS THERE IS NO INDEX

SELECT TABLE1.id, TABLE1.EVENTS,
TABLE2.id AS Expr1,
TABLE2.EVENTS AS Expr2
FROM
TABLE1
INNER JOIN
TABLE2
ON TABLE1.id = TABLE2.id

--WHAT IF WE CREATE AN INDEX BOTH TABLES

USE [AdventureWorks2016CTP3]
GO
CREATE NONCLUSTERED INDEX [TEST]
ON [dbo].[TABLE2] ([id])
INCLUDE ([EVENTS])

USE [AdventureWorks2016CTP3]
GO
CREATE NONCLUSTERED INDEX [TEST]
ON [dbo].[TABLE1] ([id])
INCLUDE ([EVENTS])
GO


--EXECUTE AND VIEW AFTER CREATION OF INDEX.  IT SHOULD BE NESTED JOIN

SELECT TABLE1.id, TABLE1.EVENTS,
TABLE2.id AS Expr1,
TABLE2.EVENTS AS Expr2
FROM
TABLE1
INNER JOIN
TABLE2
ON TABLE1.id = TABLE2.id
WHERE TABLE1.ID = 165804
```

```
--EXAMPLE OF A MERGE JOIN

--The merge join requires both inputs to be sorted on the merge columns
--The merge join is fast because it requires that both inputs are already
sorted
--Because the rows are pre-sorted, a Merge join immediately begins the
matching process.
--If the rows match, that matched row is considered in the result-set
--A Merge join performs better when joining large input tables

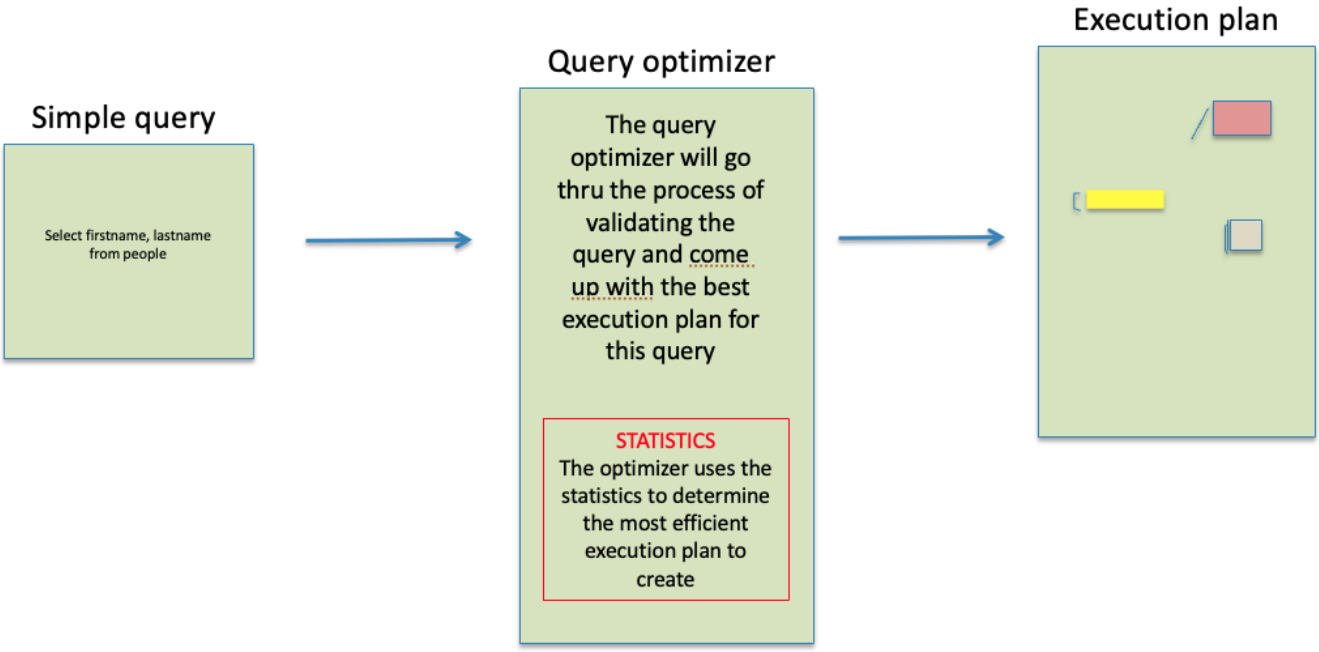

SELECT H.CustomerID, H.SalesOrderID, D.ProductID, D.LineTotal
FROM Sales.SalesOrderHeader H
INNER JOIN Sales.SalesOrderDetail D ON H.SalesOrderID = D.SalesOrderID
WHERE H.CustomerID > 100



--EXAMPLE OF COMPUTER SCALAR OPERATOR  -- CREATED WHEN YOU HAVE A NEW
COMPUTER VALUE FROM EXISTING VALUES


SELECT
OrderQty
FROM            Sales.SalesOrderDetail
WHERE SalesOrderID = '43659'
GROUP  BY SalesOrderID, OrderQty
```

Statistics



- When the optimizer is given a set of query commands, the optimizer will select the best and most efficient way to execute that query.
- To return the most efficient execution plan, the optimizer will rely on **statistics** on the table and columns to attain that result.
- So, statistics are objects that contain statistical information about the distribution of values in one or more columns of a table or indexed view.
- The Query Optimizer uses these statistics to estimate the number of rows or the cardinality returned in the query result.
- The Query Optimizer uses this cardinality estimates to choose either the index seek operator instead of the more resource-intensive index scan operator and return the most efficient execution plan.
- SQL Server automatically creates statistics when you **create a table with PK** Or when you create an index on an existing table
- When you rebuild an index it automatically creates new statistics
- By default, SQL Server has the auto create statistics enabled.

```
--when execution plans are created for they are stored in the cache for
faster retrieval

--Using DBCC FREEPROCCACHE to clear ALL cache - NEVER do this in
production but you can delete specefic plans if needed (espicially ones
that are dublicated because of an empty space)

DBCC FREEPROCCACHE

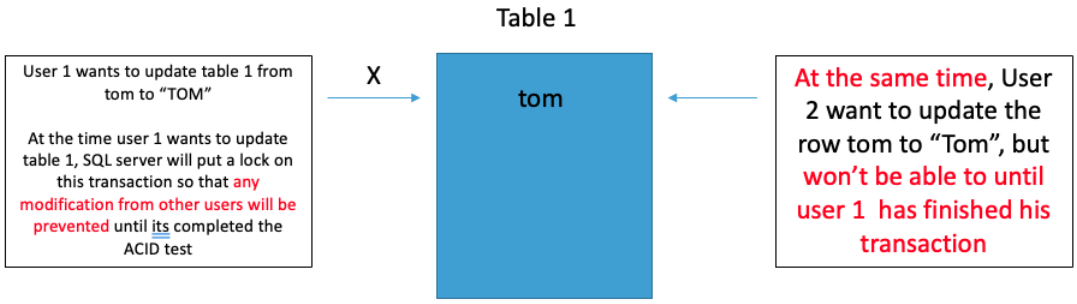
--Using DBCC FREEPROCCACHE to clear specific execution plans from the
cache

DBCC FREEPROCCACHE
```

```
(0x06000500304A1608D05E0EFEE701000001000000000000000000000000000000000000000000000000);
```

Locking

- SQL Server in an environment that has multiple users needing to access the same data simultaneously.
- SQL Server must somehow control how these reads and writes to the database are carried out.
- In a multi-user environment, locking is the process of ensuring the integrity of the data is maintained in the database
- It does this by forcing every SQL Server transaction to pass the ACID test
- note: Consistency. A transaction either creates a new and valid state of data, or, if any failure occurs, returns all data to its state before the transaction was started
- Locking can be applied to rows, tables, indexes, databases
- Locks occur during reads, inserts, updates, deletes

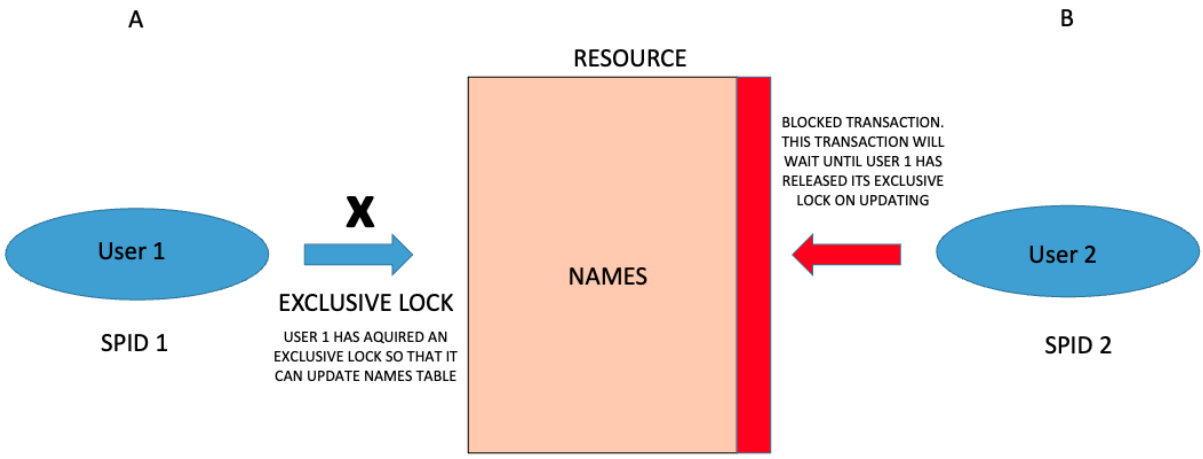


Types of Lock Modes

There are different lock modes :

- Exclusive (X):
  - Exclusive locks are used to lock data being modified by one transaction thus preventing modifications by other concurrent transactions
- Shared (S)
  - Shared locks are held on data being read. While a shared lock is being held other transactions can read but can't modify locked data
- Update (U)
  - Update locks are a mix of shared and exclusive locks. The update lock itself can't modify the underlying data. It has to be converted to an exclusive lock before the modification takes place
- Intent (I)
  - Intent locks are a means in which a transaction notifies other transaction that it is intending to lock the data

Blocking



Blocking occurs when one SPID (SPID 1) holds a lock on a specific resource and a second SPID (SPID 2) attempts to acquire a conflicting lock type on the same resource

```
--USER 1 EXECUTES AN UPDATE STATEMENT CAUSING AN EXCLUSIVE LOCK, BUT NOT
COMMITING THE TRANSACTION

BEGIN TRAN
UPDATE TABLEA
SET FN = 'TOM'
WHERE FN = 'BOB'

COMMIT TRAN  --<< NOTE NOT COMMITING THE TRANSACTION. AS SUCH, THE LOCK IS
NOT RELEASED


SELECT * FROM TABLEA


--NOW ANOTHER USER CALLED MARY TRIES TO UPDATE THE SAME TABLE WHILE THE
TRANSACTION IS STILL OPEN

--NOTICE THAT THE EXCUTION IS NOT STOPPING

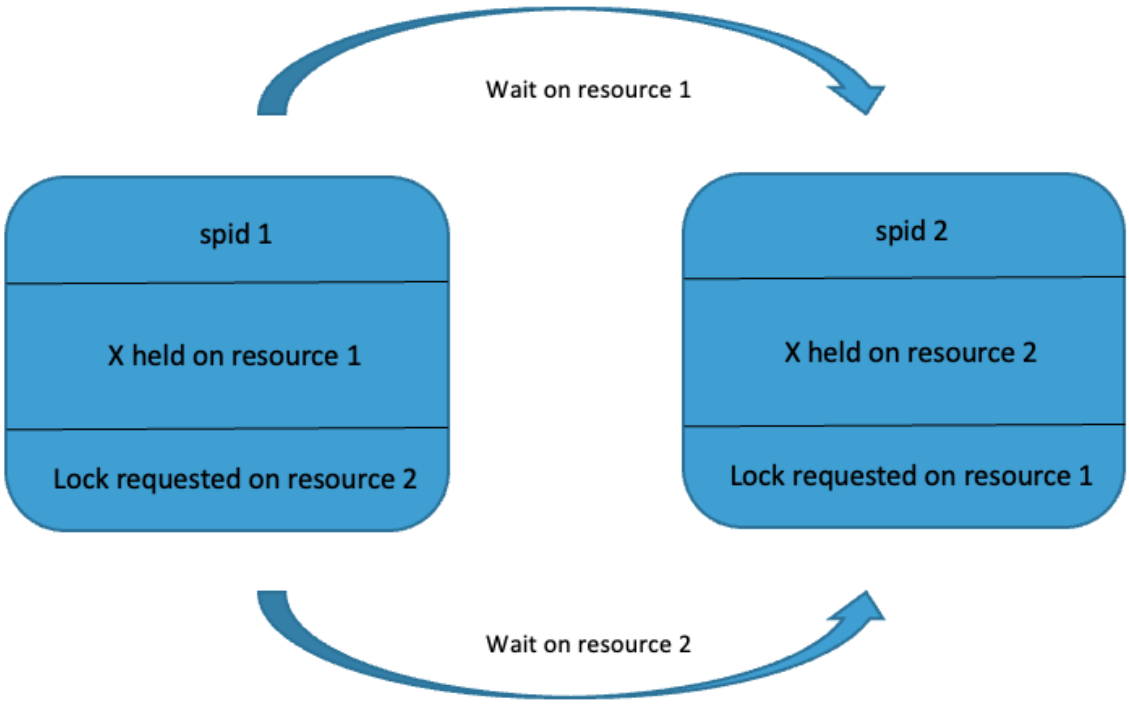
USE AdventureWorks2016CTP3
GO

BEGIN TRAN
UPDATE TABLEA
SET FN = 'SAM'
WHERE FN = 'TOM'
COMMIT TRAN

--NOW USER 1 COMMIT AND NOTICE THE RESULT.  SHOULD BE MARYS UPDATE TO SAM
```

Deadlock

- The blocking had to WAIT for user 1 to finish the update before user 2 could update the table, but the SQL server DID NOT KILL the session.
- Deadlock occurs when two or more transactions have a resource locked and the other session is requesting the locked session
- In the example below, spid 1 has a (X) on resource 1, spid 2 has a (X) on resource 2 and both are requesting locks on each others' resources: this circular chain will generate the deadlock.
- This is not be resolved by waiting
- In cases like this, SQL server detect deadlocks and declares one of the process as deadlock victim and kills that process.



Data defragmentation

Data defragmentation is a process used to reorganize data within a database to improve performance and optimize data retrieval. Over time, as data is continuously inserted, updated, and deleted, it becomes fragmented—meaning the data is scattered across different physical locations on the disk. This fragmentation leads to slower query performance and response times because the database has to access multiple locations to retrieve the complete data.

Why Data Fragmentation Occurs:

- Data Growth: As data grows and changes, the original data blocks can become scattered across the storage space.
- Frequent Modifications: Regularly updating, inserting, and deleting data can cause gaps or leave data out of sequence.
- Index Fragmentation: Indexes can become fragmented when the physical order of the index pages does not match the logical order, making it harder for the database to retrieve data efficiently.
    - Logical Order: Refers to the order in which the index organizes data based on the key values. For example, if an index is created on a column like LastName, the logical order would be alphabetical.

- Physical Order: Refers to how the data is actually stored on the disk. Over time, as data is inserted, updated, or deleted, the physical sequence of data pages can become disorganized.

Benefits of Data Defragmentation:

- Improved Query Performance: Queries run faster as data is stored contiguously, and indexes are optimized.
- Reduced Disk I/O: Less disk reading and writing are required to access data, resulting in lower latency.
- Better Storage Utilization: Consolidates storage space and reduces fragmentation, making better use of the available disk.

Defragmentation steps

Rebuilding and reorganizing indices

- Rebuilding: Drops and recreates the index, which can be more effective for severely fragmented indexes.
- Reorganizing: Physically reorganizes the leaf-level pages of the index, suitable for moderate fragmentation.

```
ALTER INDEX ALL ON employees REBUILD;
-- or
ALTER INDEX ALL ON employees REORGANIZE;
```

Shrink or Compact the Database:

- Shrinking a database reclaims unused space by rearranging data pages to remove gaps.

Update Statistics:

- Helps the query optimizer make better decisions by providing more accurate data about data distribution.

```
UPDATE STATISTICS employees;
```

Partitioning Tables:

- Split large tables into smaller, more manageable pieces based on specific criteria (e.g., date ranges).

Regular Maintenance Plans:

- Schedule periodic index rebuilding or reorganizing.
- Perform database shrink operations during off-peak hours.

Tools for Defragmentation:

- SQL Server Management Studio (SSMS): Provides built-in options for index maintenance and database shrinking.
- Oracle's Automatic Workload Repository (AWR): Helps monitor and analyze performance to decide when to defragment.
- MySQL's OPTIMIZE TABLE: Reorganizes tables and reclaims unused space.

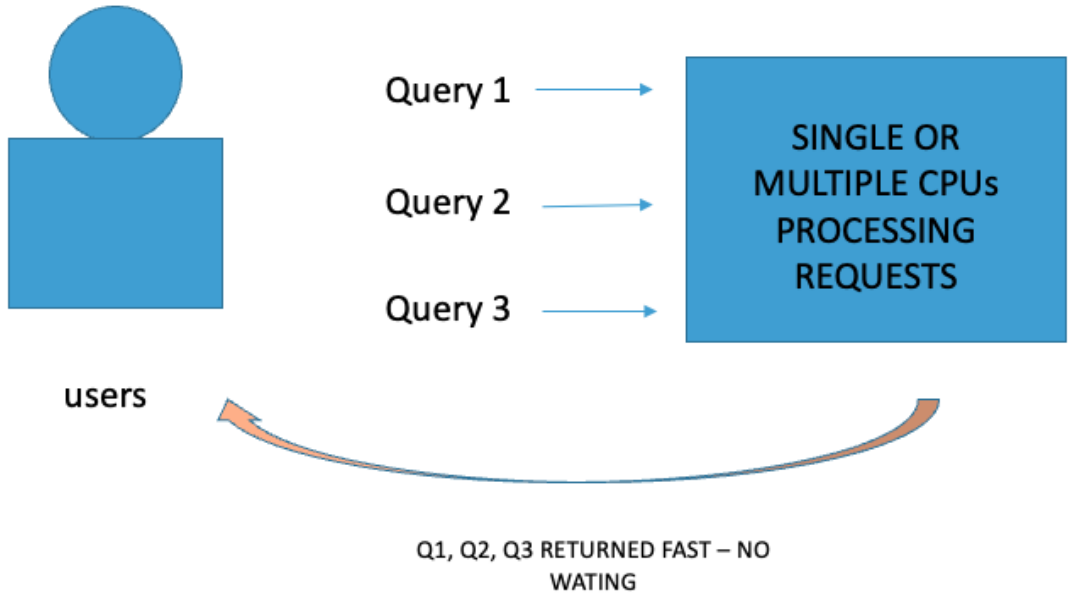
Automatic Workload Repository (AWR)

Automatic Workload Repository (AWR) is a feature of Oracle's database management system that plays a crucial role in monitoring and optimizing database performance. AWR collects, processes, and stores database performance statistics, allowing database administrators (DBAs) to analyze performance trends and resolve issues.

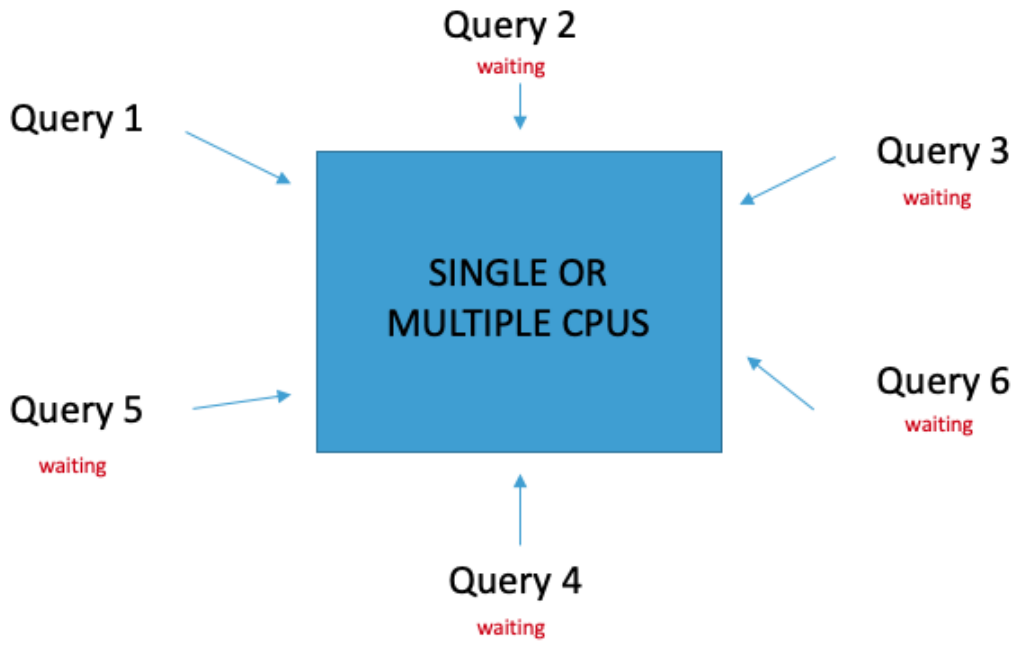
Wait Statistics

When a query is sent from an application they must `compete` for the limited CPU processor. On a single core CPU with little transactional activity, accessing the CPU for processing may not be a problem

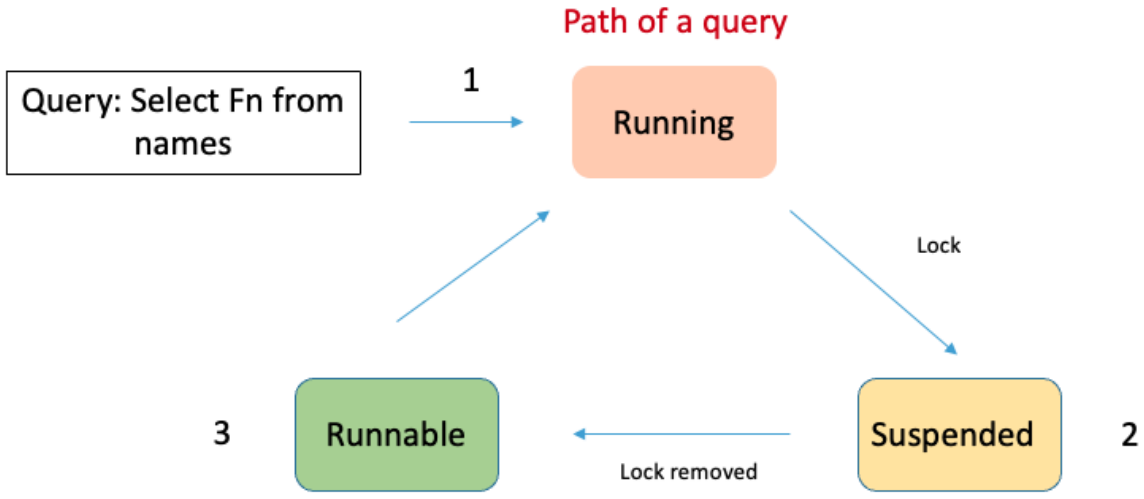
If the query is forced to wait for a resource, SQL Server records that wait stats and wait types.



- However, on a very busy transactional database, where you have thousands of users requesting their query to be processed this can become an issue. As these requests ‘compete’ for the limited CPU resource, WAITING for the CPU to process the request can become an issue. And the CPU can be taxed.
- There can be many factors causing this waiting, such as locking, poor index, badly designed query and more.



As seen on previous slide, every time an applications sends a request to the CPU for processing, the query always has to wait. This waiting can be tracked by wait statistics. (Generally this wait time is 4 ms). The best way to demonstrate this wait process is via the diagram below. As you can see, there are three states that a query can be in: Running, Suspended, and Runnable



1. When a query is being processes by the CPU it is said to be in a running state. (Ideal position)
2. If there is a lock on the query or a missing resource the query needs, then the query moves to a state suspended and moves to the queue for waiting for the resource to be obtained
3. When the missing resource the query needs is provided (lock is removed) it goes into runnable state, and the entire cycle repeats for query and all other queries

Wait types

Commonly encountered wait types include:

- **ASYNC_NETWORK_IO**: SQL Server holds data in the output buffer until it receives an acknowledgement from the client that it has finished consuming that data. ASYNC_NETWORK_IO is an indication that your client application is not able to efficiently retrieve the data it needs from the system. You may also see this wait if there are fundamental issues with your network, which can lead to long waits while the data is processed by the client and then signals returned back to the server.
- **PAGEIOLATCH_EX**: A query is waiting for exclusive write access to a page in order to add data to the page, but the page is not currently in memory and has to be loaded from disk.

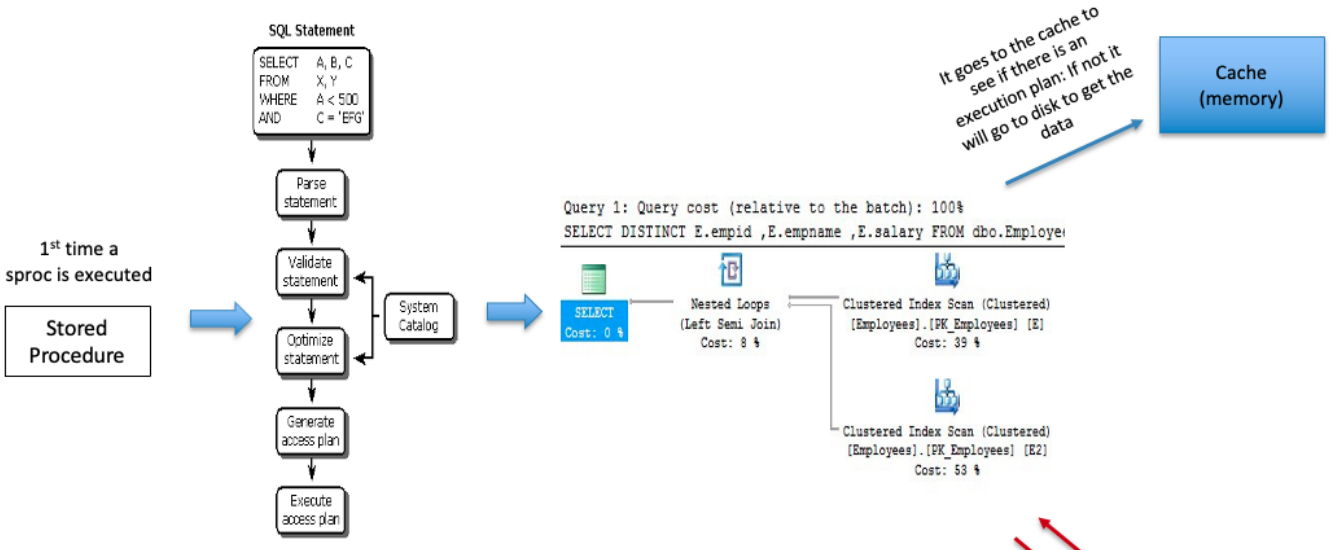
Compilation and Recompilation

A compilation is the process when a stored procedure's query execution plan is optimized by the optimizer. Once the optimizer completes, the compiled execution plan is saved in the cache.

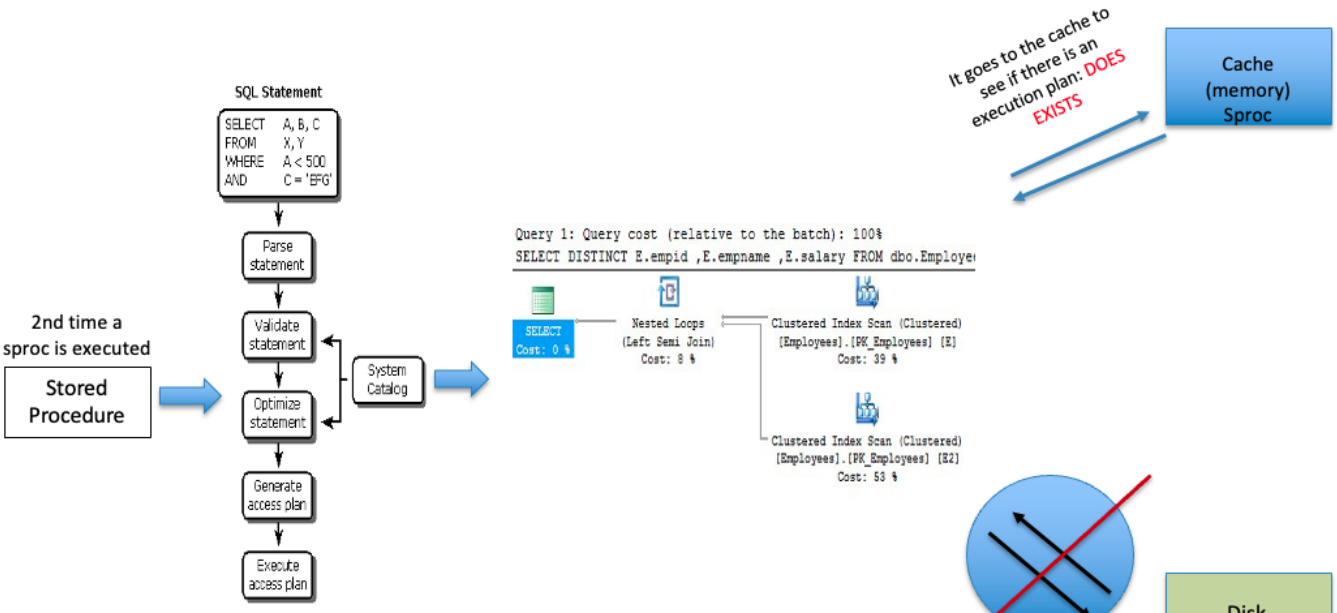
The purpose of the Query Optimizer

- A query optimizer is a SQL component that analyzes SQL queries and generates multiple execution plans and then selects the most efficient plan
- Each of these SQL statement, be it simple or complex, requires minimal use of valuable resources, such as disk reads, CPU usage and server memory

Example of a sproc compiling and execution plan



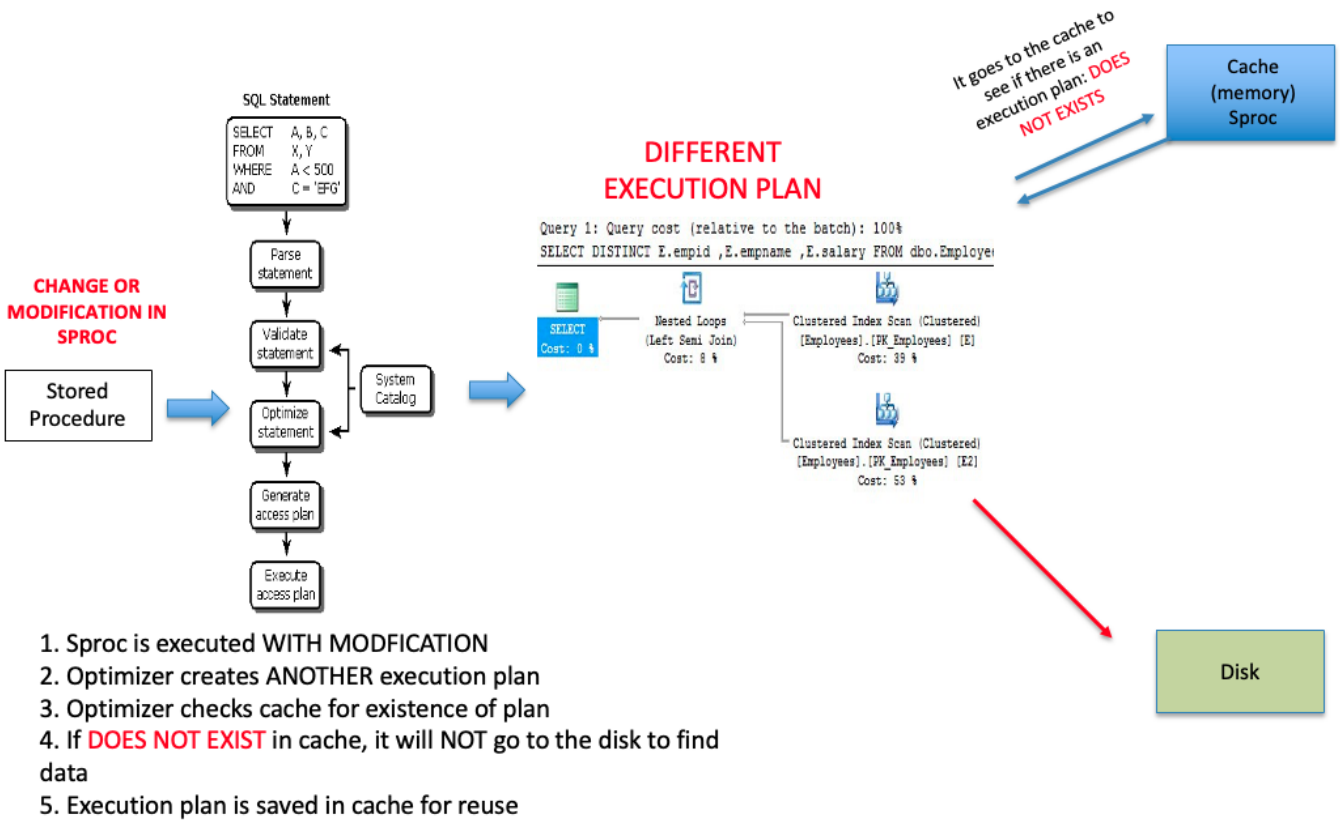
1. Sproc is executed
2. Optimizer creates an execution plan
3. Optimizer checks cache for existence of plan
4. If not in cache, it will go to the disk to find data
5. This first time creation of an execution plan is compiling



1. 2ND time Sproc is executed
2. Optimizer creates an execution plan
3. Optimizer checks cache for existence of plan
4. If DOES EXIST in cache, it will NOT go to the disk to find data
5. Execution plan is saved in cache for reuse

Recompilation

- A recompilation is the same process as a compilation, just executed again
- Recompilation occurs when the database structure or data change significantly causing a new query execution plan to be reproduced
- This recompilation degrades SQL Server performance as it's not using the cached execution plan
- Queries are reused, but their query execution plan is not
-



What is the plan cache?

- The part of the memory pool that is used to store **execution plans** is referred to as the **plan cache**. There are many type of plan cache. The one we will focus on is the SQL PLAN
- Plans are cached for possible reuse opportunities. If a query plan is not even cached, its reuse opportunity is zero. Such a plan will be compiled every time it is executed, resulting in poor performance.

Tempdb configuration

- One of the poor performance objects may the tempdb!
- The tempdb system database is a global resource that is available to all users (as such it can become very busy)
- Its purpose is to hold temporary objects and have certain action performed in the tempdb such as:
  - Global or local temporary tables
  - Temporary stored procedures
  - Table variables
  - Cursors
  - Results for spools or sorting
  - Online index operations
- Tempdb is re-created every time SQL Server is started
- **Backup and restore operations are not allowed on tempdb**
- The point is that the tempdb is a very busy database and must be configured properly for optimal performance

Setting up tempdb

- Out of the box we may get into issues of poor performance because of the default settings.
- The following list is something we must do asap:
    - Default size change for tempdb
    - Default file numbers change for tempdb
    - Finding cpu core number for tempdb files
- Set the file growth increment to a reasonable size to avoid the tempdb database files from growing by too small a value.
- If the file growth is too small, compared to the amount of data that is being written to tempdb, tempdb may have to constantly expand. This will affect performance.

File number rule: Tempdb files based on number of CPU cores (SQL 2016)

- When you have less (or equal) than 8 CPU cores, you will get as many TempDb data files as you have CPU cores. (4cpu cores = 4 temp files)
- If you have more than 8 CPU cores, you will get 8 TempDb data files out of the box.

Ranking and Partitioning

What Is RANK()?

In simplest terms, RANK() is a window function.

- The SQL window functions are similar to aggregate functions in that they are applied to a group of rows. One big difference: window functions **keep details of the individual rows**, unlike aggregate functions with GROUP BY.
- As the name suggests, it ranks data. This makes it a ranking window function, along with `DENSE_RANK()` and `ROW_NUMBER()`.
- When using any of these window functions, it must be accompanied by an OVER (ORDER BY) clause. The OVER() clause is mandatory for any window function. It's what turns a "regular" function into a window function.
- For these ranking functions, the ORDER BY in parentheses defines the order in which ranking is done. This order may be ascending or descending. Keep in mind that it does not affect the order of the rows in the result; that is done with an ORDER BY at the end of the query.

PARTITION BY

- How RANK() Works With OVER (PARTITION BY) The PARTITION BY clause divides data into partitions or subsets. When used with RANK(), this means the data is ranked within the partition. When it reaches the second partition, the ranking is reset to start from one.

Example

Our database has a table named magazine with data in the following columns: id (primary key), name, category, and price.

id	name	category	price
105	Country Living	lifestyle	1.70
108	News Magazine	news	3.35
115	Decibel	music	6.50
123	Drum Magazine	music	6.50
145	Sunset	lifestyle	12.00
155	World	lifestyle	9.50
158	Keyboard	music	8.45

```
docker run --name postgres-db -e POSTGRES_USER=myuser -e
POSTGRES_PASSWORD=mypassword -e POSTGRES_DB=mydatabase -p 5431:5432 -d
postgres
```

```
CREATE TABLE magazine (
    id SERIAL PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    category VARCHAR(50) NOT NULL,
    price NUMERIC(5, 2) NOT NULL
);
INSERT INTO magazine (name, category, price) VALUES
('Country Living', 'lifestyle', 1.70),
('News Magazine', 'news', 3.35),
('Decibel', 'music', 6.50),
('Drum Magazine', 'music', 6.50),
('Sunset', 'lifestyle', 12.00),
('World', 'lifestyle', 9.50),
('Keyboard', 'music', 8.45);
```

Let's group magazines according to category and rank each group in descending order by price. Display the ranking position, name, category, and price for each magazine.

```
SELECT id, name, category, price, RANK() OVER(PARTITION BY category ORDER
BY price desc) as rank FROM magazine
```

- SQL's `RANK()` function allows us to add a **record's position** within the result set or within each partition. In our example, we rank rows within a partition. `RANK()` is a window function used to assign a rank to each row within the result set.

- `OVER`: Specifies the window function parameters.
- The `OVER()` clause **always** comes after `RANK()`. `OVER()` **must** contain an `ORDER BY` clause. If you're returning ranks within a partition, place a `PARTITION BY` clause inside the `OVER()` clause.
- `PARTITION BY` category: Divides the data into partitions (groups - like group by) based on the category column. This means the ranking will restart for each different category.

ROW_NUMBER vs RANK vs DENSE_RANK

id	name	category	price
1	Magazine A	lifestyle	12.00
2	Magazine B	lifestyle	9.50
3	Magazine C	lifestyle	9.50
4	Magazine D	lifestyle	1.70

ROW_NUMBER()

- Assigns a unique number to each row within a partition, based on the specified order.
- If there are ties, it still gives each row a unique sequential number.
- If two rows have the same price, they will still receive different row numbers (e.g., 1, 2, 3). | id | name | price | row_number | |-----|------------------|-------|------------| | 1 | Magazine A | 12.00 | 1 | | 2 | Magazine B | 9.50 | 2 | | 3 | Magazine C | 9.50 | 3 | | 4 | Magazine D | 1.70 | 4 |

RANK()

- Ranks rows within a partition with gaps in the rank sequence if there are ties.
- If two rows tie for the same rank, they will have the same rank, but the next rank will skip over the number of ties.
- If two rows tie for rank 2, the next rank will be 4 (i.e., 1, 2, 2, 4). | id | name | price | rank | |-----|------------------|-------|------| | 1 | Magazine A | 12.00 | 1 | | 2 | Magazine B | 9.50 | 2 | | 3 | Magazine C | 9.50 | 2 | | 4 | Magazine D | 1.70 | 4 |

DENSE_RANK()

- Similar to RANK(), but without gaps in the sequence when there are ties.
- If two rows tie for the same rank, they will have the same rank, and the next rank will continue sequentially.
- If two rows tie for rank 2, the next rank will be 3 (i.e., 1, 2, 2, 3). | id | name | price | dense_rank | |-----|------------------|-------|------------| | 1 | Magazine A | 12.00 | 1 | | 2 | Magazine B | 9.50 | 2 | | 3 | Magazine C | 9.50 | 2 | | 4 | Magazine D | 1.70 | 3 |